UCLouvain

epl

École polytechnique de Louvain

Automated Carbon Footprint Estimation of Food Products

Author: **Maxime Devillet**
Supervisor: **Tom Barbette**
Reader: **Alessandro Maggi**
Academic year 2024–2025
Master [120] in Computer Science

Abstract

This thesis presents a software solution designed to automatically estimate the carbon footprint of food products based on grocery receipts. The aim is to develop a reliable tool that could eventually be used to print carbon footprint information directly on receipts, helping consumers understand the environmental impact of their daily purchases. Existing tools often rely on generic or incomplete data, limiting their precision and relevance. This project addresses these limitations by developing a complete pipeline that extracts relevant information from receipts, matches products with environmental databases such as Agribalyse and OpenFoodFacts, and calculates greenhouse gas emissions for each stage of the product life cycle: agriculture, transport, and packaging. Natural language processing techniques and similarity algorithms are used to improve product matching. The methodology incorporates factors such as seasonality, country of origin, and transport mode to produce more accurate estimates. The system also falls back on financial emission factors when specific data is unavailable. This approach enables a more transparent and detailed estimation of the carbon footprint, supporting consumers in making more sustainable choices.

Acknowledgements

First, I would like to express my sincere gratitude to Professor Tom Barbette for supporting my thesis topic and for agreeing to supervise a subject that was especially meaningful to me. I am also grateful for his guidance throughout the year, particularly through our weekly meetings, which helped me quickly identify and correct mistakes, making steady progress.

I would also like to thank Alessandro Maggi for his valuable assistance and continued support throughout the year, as well as for his insightful guidance while I was writing this thesis.

My thanks also go to Maxime Vanliefde and Nikita Tyunyayev for their involvement during the second part of this project and for the many helpful suggestions they provided for my research.

Furthermore, I would like to thank the readers and members of the jury, in particular Achille Morenville, for taking the time to read and evaluate my work.

Finally, I would like to thank my family for their unwavering support throughout this journey. I am especially grateful to Emile Villette, who was always there to offer advice, and to Silvia Spasova, whose presence and encouragement motivated me to give my best this year. I want to thank Alix Temmerman and Antoine Dumoulin for their proofreading and advice on this thesis.

I would also like to acknowledge the use of ChatGPT and Grammarly, which supported me in improving the scientific formulation and clarity of my writing throughout this Master thesis.

Contents

1 Introduction **1**
1.1 Context . . . 1
1.2 Problem Statement . . . 1
1.3 Motivation . . . 2
1.4 Approach . . . 2
1.5 Document Structure . . . 2

2 State of the Art and Related Work **4**
2.1 Background Concepts . . . 4
2.1.1 Greenhouse Gases and CO2eq . . . 4
2.1.2 Greenhouse Gas Calculation Standards . . . 5
2.2 State of the Art . . . 7
2.2.1 Environmental Databases . . . 7
2.2.2 Comparison of Environmental Databases . . . 10
2.2.3 Text Preprocessing . . . 10
2.2.4 Similarity Algorithms . . . 12
2.2.5 Distance Estimation Methods . . . 18
2.3 Related Work . . . 19
2.3.1 Existing Solutions . . . 19
2.3.2 Limitations of Existing Solutions . . . 21
2.3.3 Positioning of This Work . . . 21

3 Matching and Data Enrichment Pipeline **23**
3.1 Input Collection and Data Enrichment . . . 23
3.1.1 Matching and Data Enrichment Strategy . . . 24
3.2 Text Preprocessing and Standardisation . . . 26
3.2.1 Text Cleaning . . . 26
3.2.2 Linguistic Normalization Techniques . . . 26
3.3 Product Matching Pipeline . . . 28
3.3.1 Candidate Refinement Techniques . . . 28

3.3.2 Similarity Algorithms 29

4 Carbon Footprint Estimation Pipeline 31
4.1 Overall Architecture 31
4.2 Fallback Estimation for Missing Data 32
4.2.1 Monetary-Based Emission Estimation 32
4.3 Agricultural Emissions Estimation 32
4.3.1 Algorithm Description 33
4.3.2 Seasonality Effects 34
4.4 Transport Emissions Estimation 35
4.4.1 Algorithm Description 36
4.4.2 Transport Mode Selection 37
4.4.3 Determining Product Origin 38
4.4.4 Emission Factors by Modes 39
4.5 Packaging Emissions Estimation 43
4.5.1 Algorithm Description 44
4.5.2 Unknown Composition 45
4.5.3 Known Composition 46

5 Evaluation 47
5.1 Threats to Validity 47
5.2 Experimental Results 48
5.2.1 Matching Accuracy 50
5.2.2 Execution Time Analysis 52
5.2.3 Impact of Keyword Scoring 54
5.3 Threshold Calibration 56
5.3.1 Jaro-Winkler and Similarity Threshold 56
5.3.2 Precision vs. Recall Trade-off 56
5.3.3 Analysis of Matching Limitations 57
5.4 Algorithm Selection 59
5.4.1 Best-Performing Approach 59
5.4.2 Conclusions from Previous Subsections 60

6 Use Case Demonstration 61
6.1 Input Structure and Scenarios 61
6.2 Case 1: Minimal Information 62
6.3 Case 2: Barcode Only 64
6.4 Case 3: Complete Input 66
6.5 Case 4: Price Only 69
6.6 Results Comparison Summary 71

7 Limits and Future Work **72**

8 Conclusion **74**

Chapter 1

Introduction

1.1 Context

Since the 19th century, the overall use of fossil fuels has improved living standards but has also caused a significant increase in greenhouse gas emissions, leading to global climate disruption. To limit global warming to 1.5–2°C, the 2015 Paris Agreement commits signatory countries to reduce their emissions. Agriculture and food systems are responsible for approximately 22% of global emissions, making them a central focus of climate action [1].

One significant barrier to effective transition lies in the lack of transparent and accessible information regarding the environmental impact of daily consumption habits. In particular, food-related emissions are difficult to estimate at the individual level due to the complexity of supply chains and variability of products [2].

1.2 Problem Statement

Existing carbon footprint estimation tools, such as Eco-score [3], Greenly [4], or OpenFoodFacts [5], either provide estimations based on average product categories or cover only a limited selection of products. These approaches often overlook contextual factors, such as product origin, transportation mode, seasonality, or packaging material, which can significantly impact a product's environmental footprint.

Thus, there is a need for a tool that can estimate the carbon footprint of food products in a precise and scalable way, directly from retail data such as receipts.

1.3 Motivation

"You can't change what you can't measure." [2] This quote highlights the importance of practical and accurate tools to enable sustainable decision-making. By providing consumers with automated estimations of the climate impact of their purchases, such tools can raise awareness and encourage more environmentally responsible choices. Furthermore, this information can be integrated directly into point-of-sale systems or online platforms, allowing carbon footprint data to be printed on receipts in real-time.

1.4 Approach

To accomplish these objectives, the proposed system follows a modular and sequential architecture. It begins with the normalisation and preprocessing of text extracted from receipts. Next, the system tries to match each product with entries in external environmental databases using natural language processing tools and similarity algorithms. Based on the retrieved information, the system estimates the carbon footprint for each product, adjusting the calculation depending on the availability of contextual data. When information is missing, fallback mechanisms are used to ensure that a plausible estimation is still provided.

The solution relies on openly accessible data sources, including "Agribalyse" [6], OpenFoodFacts [7], and "Base Empreinte" [8]. It also integrates natural language processing techniques via tools such as spaCy [9], combined with similarity measures like Jaro-Winkler [10] or Cosine similarity [11], to ensure both robustness and precision in the product matching process.

1.5 Document Structure

The remainder of this thesis is structured as follows:

Chapter 2: Background and Related Work introduces the fundamental principles behind carbon footprint estimation, including key greenhouse gases, life cycle assessment (LCA), and the main international standards used in this domain, such as ISO 14040 and the GHG Protocol. It then presents a detailed review of existing tools and applications for estimating the environmental impact of food products, highlighting their methods, limitations, and coverage. This chapter also describes the ecological databases used throughout the project.

Chapter 3: Matching and Data Enrichment Pipeline explains the methods developed to extract and normalise product information from grocery receipts. It

introduces the product matching strategy based on natural language processing (NLP) and similarity algorithms. It demonstrates how additional data is retrieved from external databases to enrich incomplete inputs, such as missing origin or weight. It defines the complete pipeline that prepares product data before emission estimation.

Chapter 4: Carbon Footprint Estimation Pipeline describes how the system calculates greenhouse gas emissions for each product. It details the estimation algorithms for various life cycle stages, including agricultural production, transportation, and packaging. The chapter also explains how fallback mechanisms are applied when data is missing, such as using emission factors based on product price.

Chapter 5: System Evaluation presents the evaluation methodology and experimental results used to assess the accuracy and performance of the product matching and estimation pipeline. It includes matching precision tests, timing analysis, and optimisation of algorithm parameters. Here, we will also discuss the main limitations of the current approach and their potential impact on the results.

Chapter 6: Use Case Demonstration shows how the system behaves with different levels of input completeness. It presents multiple real-world scenarios using receipt examples, from cases where only minimal information is available, such as just the product name or price. To fully detailed entries. For each case, the resulting carbon footprint estimation is explained and compared.

Chapter 7: Conclusion and Future Work summarises the key contributions of the thesis and reflects on how the initial objectives have been met. It discusses current limitations, including the absence of tests on real scanned receipts. It proposes several directions for future research and development, such as utilising more advanced matching models and integrating with commercial systems.

Chapter 2

State of the Art and Related Work

This chapter aims to provide the scientific and technical foundations necessary to understand the design choices made in this thesis. It first introduces the main concepts and standards related to carbon footprint estimation, including greenhouse gases and life cycle assessment (LCA). It then presents the environmental databases and computational tools used to support data extraction and emission calculations.

The second part of the chapter focuses on natural language processing techniques and similarity algorithms used for product name matching, a central step in the system pipeline. Next, it explains how transport distances are estimated based on product origin.

Finally, the chapter reviews existing applications in the field and identifies their limitations. This structure highlights the added value and originality of the proposed approach.

2.1 Background Concepts

In this section, we will discuss the essential concepts necessary to understand the carbon footprint estimation methodology. This section includes the definition of carbon footprint, the use of CO2 equivalent as a standard unit, and the principles of Life Cycle Assessment (LCA) [1], [12], [13].

2.1.1 Greenhouse Gases and CO2eq

The carbon footprint refers to the total amount of greenhouse gases emitted directly or indirectly to conceive a product or process. We will express it in carbon dioxide equivalent (CO2eq). [1]

Several types of gases contribute to climate change, including Carbon dioxide (CO2), the most popular, Methane (CH_4), Nitrous oxide (N2O), etc.

Each of these gases has a different global warming potential (GWP), which represents the amount of heat the gas traps in the atmosphere compared to CO2 over a 100-year period.[1]

CO2 Equivalent Calculation

To simplify impact assessments, all emissions are converted into an equivalent amount of CO2, known as CO2 equivalent (CO2eq):

$$\mathrm{CO_2eq} = \sum_i (m_i \times \mathrm{GWP}_i)$$

m_i: is the mass of greenhouse gas i
GWP_i: is its Global Warming Potential.

In our case, we will use the kilograms of carbon dioxide equivalent unit (kg CO2eq) to represent the carbon footprint equivalent emitted for the production, transportation, storage, and other related activities associated with 1 kg of product.

2.1.2 Greenhouse Gas Calculation Standards

In this section, we analyse the calculation methods and standards currently used in the scientific literature to provide a comprehensive overview of existing approaches for assessing the carbon footprint of food products. These methods are compared to identify their respective advantages and limitations, depending on the specific application context.

Life Cycle Assessment

(LCA) [14] LCA is the foundational methodology used to evaluate the environmental impacts of products or services across all stages of their life, from raw material extraction to end-of-life disposal. In the food sector, it provides a robust and systematic approach to assess the full range of emissions generated by food production, processing, packaging, transport, consumption, and waste management.

This methodology is formally defined by two key international standards: ISO 14040 [12], which establishes the overall structure and phases of a Life Cycle Assessment (LCA), and ISO 14044 [13], which provides detailed requirements regarding data quality, allocation, and reporting. These standards divide the LCA process into four main phases: inventory analysis, impact assessment, interpretation,

and reporting. In food systems, additional factors such as enteric fermentation, cold-chain logistics, seasonality, transport distance, and greenhouse use are particularly relevant and can significantly affect LCA outcomes [15].

Several derived standards are based on the ISO LCA framework but adapt it to specific goals, particularly for quantifying greenhouse gas emissions (GHGs), which are the primary focus of this thesis.

ISO 14067

The ISO 14067 [16] is a specialised standard that focuses exclusively on the carbon footprint of products. It uses the same LCA structure but narrows the scope to climate change impacts only, expressed in kilograms of CO2 equivalent. It requires rigorous documentation of data sources, emission boundaries, and allocation rules, making it especially relevant for sectors aiming to reduce GHG emissions specifically [15], [17].

PAS 2050

[18] PAS 2050 is another methodology that draws on LCA principles but is designed for ease of use and faster implementation. It also focuses solely on carbon emissions and includes the full life cycle of a product from production to disposal. When specific data is unavailable, PAS 2050 allows the use of generic datasets such as ecoinvent [19]. It promotes the use of Product Category Rules (PCRs) to tailor the analysis to product types such as fresh produce. Although it is less detailed than ISO standards, PAS 2050 offers a practical approach to product-level carbon footprinting and has seen widespread adoption in the UK. However, it is less commonly used internationally than ISO 14067. [15], [17].

The GHG Protocol

[20] The GHG protocol offers a broader framework, applicable not only to products but also to entire organisations and supply chains. Developed by the World Resources Institute and the World Business Council for Sustainable Development, it is the most widely used reporting standard among large corporations. It classifies emissions into three scopes: direct emissions (Scope 1), indirect emissions from purchased energy (Scope 2), and other indirect emissions throughout the supply chain (Scope 3). While it is less product-specific than ISO or PAS standards, it provides a complete accounting framework for corporate climate reporting [4], [15], [17].

2.2 State of the Art

2.2.1 Environmental Databases

This section presents the most comprehensive and relevant environmental databases for this study. We explain their value in the context of carbon footprint estimation, while also highlighting their limitations. The scope is limited to European databases due to their higher relevance to our subject.

EcoInvent

EcoInvent [19] is a widely used environmental database in Europe that provides detailed life cycle inventory (LCI) data across multiple sectors, supporting life cycle assessment (LCA) studies with standardised and transparent datasets. Developed in Switzerland, it comprises thousands of datasets spanning a wide range of sectors, including agriculture, food production, transportation, energy, and packaging.

The database provides highly detailed and regularly updated data. Each process is documented with its inputs, outputs, and associated environmental impacts, making EcoInvent a key reference for life cycle assessment (LCA) studies. It also serves as a foundational source for many other ecological databases.

The primary advantage of EcoInvent lies in the high quality of its data and its compatibility with most Life Cycle Assessment (LCA) software, including SimaPro, OpenLCA, and GaBi. It also allows for regional customisation, such as adapting datasets to the local electricity mix, enabling more precise and context-specific assessments.

Although EcoInvent requires a paid license, selected datasets, especially transport-related emission factors, are accessible through public interfaces such as "Base Empreinte" for users based in France. This partial access helps mitigate some of the limitations for smaller research projects [21].

Agribalyse

"Agribalyse" [22] is a French environmental database developed by ADEME[1], in partnership with EcoInvent (see section 2.2.1), making it a reliable and openly accessible source. It is specialised in the agricultural and food sectors. "Agribalyse" provides life cycle assessment (LCA) data(see subsection 2.1.2) for over 2,500 food products consumed in France, encompassing a range of raw ingredients, such as wheat, milk, and meat, as well as processed foods like pizzas and ready-made meals.

[1]ADEME is a French public agency under the supervision of the Ministry for Ecological Transition [23]

The database is structured in two parts [6]: a simplified CSV file for quick reference, and a comprehensive dataset in ILCD format [2] that includes detailed calculation data. The detailed version requires specialised software such as OpenLCA or SimaPro to be read and analysed. All assessments follow ISO 14040 standards (see subsection 2.1.2).

"Agribalyse" relies on attributional Life Cycle Assessment (LCA) methodology and provides multiple environmental indicators. In addition to greenhouse gas emissions, it reports on acidification, eutrophication, land use, and water consumption. These metrics are essential for a broader understanding of the environmental footprint of food products [6].

The primary advantage of "Agribalyse" is that it provides a comprehensive, open-access database encompassing nearly all food products available in France. It reflects a commitment to transparency regarding the environmental impacts of food items.

However, "Agribalyse" provides national average values for each product category. While it encompasses general life cycle stages such as transport and packaging, it does not systematically account for specific contextual factors, including the exact country of origin, transport mode, or seasonality, except in a few cases, such as strawberries and tomatoes, where seasonal variations are explicitly reported. Furthermore, the database is focused on environmental impacts within France, which may reduce the relevance and accuracy of the data for imported items.

Base Empreinte

"Base Empreinte" [8] is an official public database managed by ADEME (see section 2.2.1). It provides emission factors and inventory datasets for a wide range of economic activities and materials, including food data. The database is designed to support carbon footprint calculations across various sectors, including transportation, packaging, energy, and manufacturing.

One of its primary uses is to provide average emission factors for standard processes, such as freight transport by various modes (truck, rail, ship, plane), electricity production, or material manufacturing. These factors are regularly updated and based on life cycle assessments (LCAs) aligned with ISO 14040 (see subsection 2.1.2) standards.

"Base Empreinte" is beneficial when we want to consider emissive factors to compute data, such as the carbon footprint of a trip or the packaging of a product.

[2]The ILCD format is a standardised data format for life cycle inventories developed by the European Commission [24].

Its data are openly accessible through an online interface and can also be downloaded in structured formats for integration into automated pipelines. Since April 2025, selected datasets from ecoinvent have been directly available within the "Base Empreinte" database for individuals located in France[3]. In particular, the transport-related emission factors were updated on April 29, 2025. These now include average values across different transport modes such as air, sea, and others.

OpenFoodFacts

OpenFoodFacts is a collaborative, open-access database derived from the mobile application of the same name. It is maintained by a global community of contributors who collect and upload data on food products sold in stores worldwide. The platform operates under an open-data model and is governed by a non-profit organisation.

The database contains detailed information on over 3 million food items, including product names, countries of origin, brands, barcodes, packaging materials, nutritional values, and many other valuable attributes. This makes it one of the most extensive publicly available food databases.

OpenFoodFacts is accessible through a web interface and an API, and the whole dataset can also be downloaded as CSV or JSON files [5]. The barcode system makes it especially useful for linking commercial products to environmental databases such as "Agribalyse" (see section 2.2.1), which is directly integrated in the API data.

Its main advantages lie in its openness and scale. However, data quality can vary due to its collaborative nature. Some entries may lack critical information, such as product weight or origin, and not all products include data on their environmental impact. Additionally, while the platform links to "Agribalyse" categories for certain products, these links are not always consistently provided or validated [5].

CarbonCloud

CarbonCloud [25] is a software platform based on a biophysical model that has been developed over more than 20 years of scientific research. The model enables rapid and accurate calculation of climate footprints.

The platform estimates a product's carbon footprint by taking into account the country of production. It accounts for national differences in production methods, including factors such as deforestation, pesticide usage, and technological advancement.

[3] As reported by ADEME on the "Base Empreinte" platform [8].

2.2.2 Comparison of Environmental Databases

Table Table 2.1 presents a summary comparison of five major environmental databases used in this study. The table highlights key characteristics, including access type, sectoral coverage, update frequency, and dataset size.

Database	Access	Sector	Updates	Size
EcoInvent	Paid	Multi-sector	Regular	18,000+
Agribalyse	Free	Food emissions	Irregular	2,500+
Base Empreinte	Free	Emission factors	Regular	1,000+
OpenFoodFacts	Free	Food products	Continuous	3M
CarbonCloud	Proprietary	Food emissions	Regular	56,000+

Table 2.1: Comparison of Key Environmental Databases (without geographic scope)

Note: *In the size column in the table, OpenFoodFacts refers to individual branded food products; other datasets list product categories or environmental models, not commercial items.*

2.2.3 Text Preprocessing

In this section, we will describe the key text preprocessing techniques applied to the data in this work. We will begin by discussing lemmatisation and stemming, two methods for reducing words to their base or root forms, and their respective advantages. Next, we will explore the use of spaCy, a powerful NLP library, which is utilised for tokenisation and lemmatisation in this project. Finally, we will examine two popular stemming algorithms, the Paice/Husk Stemmer and the Snowball Stemmer, highlighting their functionalities and relevance to the task at hand.

Lemmatization

Lemmatization [26] is the process of converting a word to its base or dictionary form, called a lemma. For example, the words *"mangeraient"*, *"mangé"*, and *"mangeons"* are all reduced to *"manger"*. This process helps group variations of a word by removing grammatical inflexions. However, the correct lemma depends on the word's role in the sentence (verb, noun, adjective), making the technique more linguistically precise but also more computationally intensive.

Stemming

Stemming [26] is a more straightforward method that cuts words to their root form without considering their grammatical function. For example, both *"mangé"* and *"mangeais"* may be reduced to *"mang"*. While this approach is less accurate than lemmatization, it is faster and often sufficient for identifying similar word patterns in noisy data such as product labels on receipts.

spaCy

spaCy [9], [27] is an open source library for Natural Language Processing (NLP) in Python. It was created by Explosion AI and offers a complete NLP pipeline, including tokenisation and lemmatisation. It provides pre-trained models for several languages, including French. We selected spaCy because it offers good support for French, fast processing, and strong community support. It also has a well-documented and straightforward API.

spaCy includes three French models of different sizes:

- `fr_core_news_sm`: a small model that is fast and light but less accurate [28];
- `fr_core_news_md`: a medium model with good performance and useful word vectors [29];
- `fr_core_news_lg`: the largest model with the most features, but slower and often unnecessary for simple tasks [30].

Paice/Husk Stemmer

The Paice/Husk stemmer [31] is a rule-based stemming algorithm designed to iteratively remove suffixes from words using a comprehensive set of rewrite rules. Unlike simple suffix-stripping methods, it allows partial matches and multiple passes over the word, making it highly flexible and more aggressive than other stemmers.

Its main advantage lies in its balance between stemming strength and linguistic precision, often resulting in fewer overstemming errors. It is frequently used for languages with complex morphology, such as French. Still, it requires custom implementation and tuning, as it is not widely supported in modern natural language processing (NLP) libraries.

Snowball Stemmer

The Snowball stemmer, also known as the Porter stemmer, is one of the most widely adopted stemming algorithms, especially in information retrieval systems. It

was designed for simplicity and speed while maintaining acceptable accuracy [31].

Implemented in many programming languages, including Python through the NLTK library, Snowball supports multiple languages, including French. It employs a sequence of rules to reduce words to a standard stem, although it is generally less complex than Paice/Husk in its reduction process.

2.2.4 Similarity Algorithms

In this section, we will describe the different similarity algorithms used in this work, their functions, and the scoring functions that enable each algorithm to determine the probability that two names match or not. We will conclude by providing a summary table of these different algorithms to better highlight their strengths and weaknesses within the context of this work.

Jaccard

The Jaccard measure compares the sets of words (or tokens) found in two strings [32]. It calculates the ratio between the number of words the strings have in common (the intersection) and the total number of unique words across both strings (the union):

$$\text{Jaccard}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

In simple terms, the more words two strings share, the higher their Jaccard score will be. This method is easy to use and works well when the order of words is not essential.

According to Hariharan, although Jaccard scores are usually slightly lower than Cosine scores (see section 2.2.4), they offer better discrimination. This means it reduces false positive cases where two unrelated products are matched incorrectly.

In our case, the simplicity of the Jaccard method can yield good results by correctly filtering items that share many words without favouring longer strings over shorter ones. However, its simplicity may cause it to miss valuable information hidden in the structure or order of the words. Still, it could provide a strong baseline for comparison with more advanced algorithms.

Dice

The Dice measure the Sørensen–Dice coefficient, which compares the number of words two strings have in common [32]. Like Jaccard (see section 2.2.4), it works with sets of words, but it gives more weight to the words they share. It doubles the size of the intersection compared to the total number of words:

$$\text{Dice}(A, B) = \frac{2 \cdot |A \cap B|}{|A| + |B|}$$

Dice gives higher importance to shared words, even when the strings have different lengths. This helps avoid giving more value to longer strings, which may have more common words just by chance.

Dice is a good middle point between Jaccard (see section 2.2.4) and Cosine (see section 2.2.4). It often yields higher scores than Jaccard and can identify more accurate matches, while still distinguishing between unrelated strings.

Cosine

The Cosine similarity method gives more importance to words that appear frequently in both strings [32]. It employs a vector-based approach: each string is transformed into a vector of numbers, where each dimension corresponds to a word in the vocabulary.

In our case, we follow the method described by Andersson [33]. He uses TF-IDF (Term Frequency–Inverse Document Frequency) to create the vectors. This one assigns a weight to each word based on:

- Term Frequency (TF): Words that appear more often in a product name are given higher weight.
- Inverse Document Frequency (IDF): Words that are rare across the whole dataset are considered more informative for matching.

This helps reduce the impact of less useful and common words and increases the importance of specific words that help distinguish products.

Once the strings are converted into vectors, the cosine similarity measures the angle between them. A slight angle (meaning a cosine value close to 1) indicates high similarity in word usage, while a value near 0 indicates low similarity.

Cosine similarity, combined with TF-IDF, helps identify product names that share uncommon but essential terms. However, it does not capture semantic similarity between different but related words..

Jaro

Jaro and Jaro-Winkler are two well-known character-based fuzzy matching methods [10]. Unlike token-based methods, they compare strings character by character, making them effective at handling typos and minor spelling errors.

The Jaro similarity measures the similarity between two strings based on three factors: the number of matching characters, the number of characters that appear in the wrong order (called transpositions), and the lengths of the strings. It is particularly useful for comparing short strings such as names. The closer the Jaro score is to 1, the more similar the two strings are.

The formula to compute the Jaro similarity between two strings s_1 and s_2 is:

$$Jaro = \frac{1}{3}\left(\frac{m}{|s_1|} + \frac{m}{|s_2|} + \frac{m-t}{m}\right)$$

Where:

- m is the number of matching characters,
- t is the number of transpositions[4]
- $|s_1|$ and $|s_2|$ are the lengths of the strings.

The final Jaro distance is computed as:

$$\text{Distance} = 1 - Jaro$$

[4]A transposition occurs when a matching character is found in both strings but not in the same position. Since each mismatch involves two characters (one in each string), the total number of transpositions is divided by 2.

Example: Jaro Similarity for "MART" and "KARMA"

The strings "MART" and "KARMA" share three matching characters: 'A', 'R', and 'M', so $m = 3$. However, 'M' doesn't appear in the same positions. Since each transposition affects one character, we divide the count by 2, resulting in $t = 0.5$. The lengths of the strings are $|s_1| = 4$ and $|s_2| = 5$.

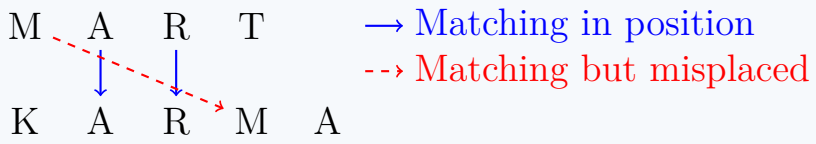

Figure 2.1: Jaro example between "MART" and "KARMA".

Plugging these values into the formula gives:

$$Jaro = \frac{1}{3}\left(\frac{3}{4} + \frac{3}{5} + \frac{3 - 0.5}{3}\right) = \frac{1}{3}(0.75 + 0.6 + 0.833) \approx 0.728$$

Jaro-Winkler

Jaro-Winkler is an enhancement of the Jaro similarity metric, designed to improve accuracy when the compared strings share a common prefix. This adjustment is particularly relevant in real-world contexts where differences often occur near the end of strings, such as in product names extracted from receipts [10].

The original Jaro metric computes similarity based on the number of matching characters and transpositions between two strings. However, it treats all parts of the string equally, regardless of character position. Jaro-Winkler improves upon this by introducing a prefix scaling factor that increases the similarity score when the beginning of both strings matches. This is especially useful in name-matching tasks, where the most informative parts often appear at the start.

The Jaro-Winkler similarity score is defined as:

$$JW = J + (l \cdot p \cdot (1 - J))$$

where:

- J is the original Jaro similarity score,
- l is the number of matching characters at the beginning of both strings (up to a maximum of 4),

- p is a scaling factor, typically set to 0.1.

Example: Jaro-Winkler Similarity for "MART" and "KARMA"

The Jaro-Winkler similarity adds a prefix bonus. Assuming a scaling factor $p = 0.1$ and common prefix length $l = 1$, we apply:

$$Jaro\text{-}Winkler = Jaro + (l \cdot p \cdot (1 - Jaro)) = 0.728 + (1 \cdot 0.1 \cdot (1 - 0.728)) \approx 0.755$$

By assigning more weight to initial matches, Jaro-Winkler can better distinguish between items with similar suffixes but distinct roots. In our case, this is crucial for accurately matching noisy or abbreviated product names found on receipts. For example, "Yaourt nature" and "Yaourt nature 4x125g" would be more correctly associated than with plain Jaro.

Therefore, Jaro-Winkler was selected over Jaro in this work because it provides improved performance in prefix-sensitive matching scenarios, offering better tolerance to end-of-string discrepancies while preserving accuracy for short, irregular entries [10].

Token-Based Filtering

A common approach is to apply a pre-filtering step that reduces the number of candidate comparisons, thereby improving the efficiency of string matching tasks. One such method is token-based filtering, which selects only those database entries that share the most relevant tokens with the input string.

Zeakis et al. [34] introduced a filtering technique based on the concept of token utility, allowing estimation of the maximum possible similarity between two sets of tokens before performing full similarity calculations. Their approach significantly reduces computational cost while preserving matching accuracy, often eliminating the most of irrelevant candidates without degrading performance.

Comparaison Table

Method	Principle	Advantages (+)	Limitations (-)
Jaccard	compares common vs. total tokens	• Simple to compute • Works with unordered words • Few false matches	• Ignores word order • Misses partial matches • Sensitive to noise
Dice	gives more weight to common words	• Simple to compute • Better match rate than Jaccard • Handles differents size of strings	• Ignores word order • Can match too loosely • Sensitive to noise
Cosine	Measures the angle between word vectors based on importance	• Highlights rare words • Good for close meanings • Works with long texts	• Needs full vocabulary • Slower to compute • Needs cleaning and preparation
Jaro	Compares character order and position	• Detects typos • Good for short words • No need for word splitting	• Weak for long texts • Fails with reordered words • No understanding of meaning
Jaro-Winkler	Jaro + extra weight for the same prefix	• Strong on brand names • Better at small mistakes	• Too focused on beginnings • Not ideal for generic texts

Table 2.2: Summary of similarity methods with their formula, advantages, and limitations

2.2.5 Distance Estimation Methods

This section presents the different algorithms used to calculate the travel distance between two geographic points, depending on the type of transportation. We begin with OpenRouteService [35], an open-source tool used to compute the shortest road route, particularly for freight transport by truck. We then introduce SeaRoute [36], which can be considered a maritime counterpart to OpenRouteService, designed to estimate the shortest sea route between two locations.

OpenRouteService

OpenRouteService [35] (ORS) is an open-source routing API developed by HeiGIT and powered by OpenStreetMap data. It provides detailed routing capabilities for various transport modes, including driving, walking, cycling, and heavy goods vehicles. In this project, ORS is used to calculate the shortest road route between two geographical points, typically from the capital city of the product's country of origin to a store location, representing the likely path of food transportation by truck.

ORS returns the shortest route and an indicative travel time based on road network data. However, these estimates do not account for real-time conditions such as traffic or weather.

A known limitation of ORS is its maximum routing range of approximately 6,000 kilometres, which is exceeded when this limit is exceeded.

SeaRoute

SeaRoute [36] is an open-source Python library designed to calculate the shortest maritime route between two geographical coordinates or ports. It operates using a predefined global maritime network and applies Dijkstra's algorithm to identify the most efficient path across sea lanes.

Unlike commercial maritime routing solutions, SeaRoute does not account for real-time navigation factors such as active shipping lanes, weather conditions, piracy risk areas, or port congestion. However, it remains a reliable and lightweight alternative for estimating large-scale maritime distances with acceptable precision.

2.3 Related Work

2.3.1 Existing Solutions

In this section, we review the state of the art regarding existing applications that estimate the environmental impact of food products. We present an overview of the main tools developed in this area, highlighting their objectives, methods, and limitations. This analysis allows us to position the specific contribution of this work in comparison to existing solutions.

Typology of Existing Tools

Several applications have been developed to help consumers estimate the environmental impact of their food purchases. These tools follow different approaches depending on the data used and the level of precision they aim to achieve. We can group them into three main categories:

- **Monetary-based estimations**, such as Greenly and BNP Paribas[37], which use the price of purchases with average emission factors per sector to compute an estimation.
- **Product-based estimations**, such as Open Food Facts (see section 2.2.1) [5] and La Fourche [38], which use life cycle assessment databases to analyse and compute a more accurate estimation.
- **Score-based summarisations**, such as Eco-score [3], which provide an overall score based on several environmental criteria like LCA, combined with a bonus-malus system that adjusts the score.

Each approach has its own advantages and limitations, which are discussed below through representative examples.

Eco-score

Eco-score [3] is an environmental labelling system that rates food products from A (low impact) to E (high impact). It is based on life cycle assessment (LCA) data from the "Agribalyse" database (see section 2.2.1). In addition to this core data, Eco-score applies a bonus-malus system that adjusts the score according to various criteria such as the origin of ingredients, type of farming, packaging materials, and impact on biodiversity.

Its main goal is to simplify environmental information for consumers and promote more sustainable choices. However, the Eco-score is calculated using average values per product category and does not take into account the specific context of purchase,

such as the exact country of origin, brand variations, or seasonality. This limits its accuracy in real-life applications.

Greenly

Greenly [37] is a mobile application that estimates the carbon footprint of consumer purchases by analysing banking transaction data. In partnership with BNP Paribas, it use average emission factors per euro spent, differentiated by sector, such as food, transport, or energy. This method is convenient, as it only requires the name of the store and the purchase price to generate an estimate, enabling large-scale deployment without needing detailed product-level data.

However, while user-friendly and straightforward, this method remains highly approximate. It cannot differentiate between two products with the same price but significantly different environmental impacts. It also does not account for specific product characteristics such as origin, packaging, or mode of transport. As a result, it is better suited for general awareness than for accurate product-level assessment.

La Fourche

La Fourche [39] is an online organic supermarket that displays the carbon footprint of its products directly on its platform in the form of a carbon rating ranging from A+ to E, indicating the relative carbon impact of the product compared to foods typically consumed by the French population [38]. The estimations rely on LCA data, mainly sourced from "Agribalyse" (see section 2.2.1), and cover multiple stages of the life cycle, including agriculture, processing, and distribution. The goal is to guide customers toward products with a lower environmental impact.

Although more detailed than transaction-based approaches, this method remains constrained by data availability. When ingredient origin or transport details are missing, estimations rely on standardized assumptions or average values from public databases. As a result, the carbon footprint displayed by La Fourche is more refined than category-level averages but still not fully individualized per product batch.

Open Food Facts Application

Open Food Facts [7] is a collaborative, open-access database that provides nutritional and environmental information about food products. It collects data on packaging, carbon footprint, product origin, and links to "Agribalyse" (see section 2.2.1). The database is widely used by applications such as Yuka [40] for barcode-based scanning and analysis.

Its key strengths are openness, scale, and community-driven updates. It contains

millions of product entries and is regularly updated. One significant advantage is that users only need a product's barcode to retrieve relevant information. However, the carbon footprint values are often based on category-level averages (e.g., "apple", "pizza") rather than brand-specific or origin-specific data. The system also has limitations when it comes to unpackaged or bulk products, which typically lack universal barcodes and are therefore harder to identify reliably.

Moreover, the presence of carbon footprint data is not guaranteed for every product. Since the platform relies on user contributions, the availability and accuracy of such information can vary significantly. As a result, some entries may lack emission estimates entirely, while others may include data of uncertain reliability.

2.3.2 Limitations of Existing Solutions

Despite their usefulness, most existing applications provide estimations that rely on average values and fail to account for context-specific factors such as seasonality, transport method, or product characteristics (see section 2.3.1). As a result, these tools often work only for a specific store or a restricted set of products (see section 2.3.1).

This limitation renders them less suitable for capturing the true variability in product emissions, which is crucial for enabling consumers to make informed and responsible choices. In practice, a more adaptive system is needed, one that can operate across a wide range of products and reflect the actual impact of individual purchases.

2.3.3 Positioning of This Work

The system developed in this thesis [41] addresses the above limitations by proposing a more accurate and transparent method. Unlike existing applications (see section 2.3.1), it enables automatic and open-access carbon footprint estimation by combining data from environmental databases with advanced text processing and similarity algorithms.

In addition to standard life cycle data, the system enhances its precision by integrating key contextual factors such as seasonality, country of origin, transportation mode, and packaging composition. This personalised estimation pipeline provides a more accurate representation of each product's carbon footprint. It is designed to be extensible, allowing for easy incorporation of future improvements as more detailed databases become available.

Moreover, the entire methodology is documented, reproducible, and built using

publicly accessible data sources, which strengthens both transparency and adaptability.

Table Table 2.3 presents a comparison of several existing tools, highlighting how our system performs in terms of both estimation accuracy and product coverage.

Application	Approach	Good Accuracy	Covers numbers of Products
Eco-score	Score based on LCA + bonus/malus	No	Yes
Greenly	Monetary-based estimation using bank data	No	Yes
La Fourche	Product-specific LCA for their catalogue	Yes	No
Open Food Facts	Barcode-based open database linked to "Agribalyse"	No	Yes
Our Approach	Context-aware estimation via data enrichment and NLP matching	Yes	Yes

Table 2.3: comparison of existing tools by estimation precision and product coverage

Chapter 3

Matching and Data Enrichment Pipeline

When extracting information from a receipt or other textual source, it is common to have only partial product data, such as an abbreviated name or missing quantity. This limitation reduces the accuracy of carbon footprint estimations. To address this, we enrich the available input using external environmental databases, notably OpenFoodFacts [7], Agribalyse [22], and "Base Empreinte" [8]. The goal is to retrieve the most relevant product information by identifying the closest match based on text analysis and contextual cues.

Unlike existing tools that rely mainly on barcodes or exact product names, such as OpenFoodFacts [7] or Eco-score [3], our method uses a combination of similarity algorithms to match products across multiple databases, even when names differ or are incomplete. This enables the system to identify relevant items that other applications would typically overlook, thereby enhancing both robustness and coverage in real-world conditions [26].

This chapter describes the complete matching and data enrichment pipeline. First, we explain how data is collected and complemented using public sources. Then, we detail the text preprocessing steps required to standardise noisy inputs. Finally, we present the matching algorithms used to associate receipt items with reference products.

3.1 Input Collection and Data Enrichment

The first step is collecting accurate and complete product information using as little initial data as possible. To achieve this, we link store-sold products with external

databases that can provide supplementary information, notably when key details such as product name, quantity, origin, or estimated CO_2 emissions are missing.

To standardise product quantity, we convert all weight values into kilograms. In cases where the amount is expressed as several items or "pcs", we currently treat this as missing data.

3.1.1 Matching and Data Enrichment Strategy

This section presents the strategy used to enrich product data extracted from receipts. We query external databases to retrieve relevant metadata since receipt information, such as missing weight, origin, or product category, is often incomplete. Figure Figure 3.1 illustrates the decision process used to access each data source.

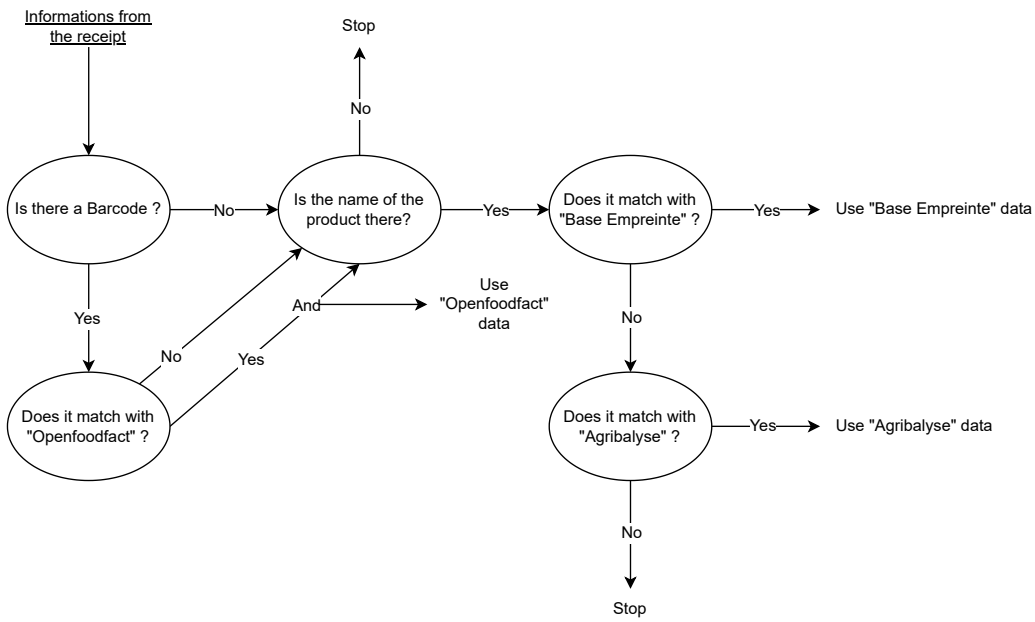

Figure 3.1: Data enrichment flow based on available input data.

We rely on three external databases: OpenFoodFacts [7], "Base Empreinte" [42], and "Agribalyse" [6]. The enrichment process follows three main steps:

Step 1: Querying OpenFoodFacts

Data Quality in OpenFoodFacts

OpenFoodFacts is a collaborative platform where most product data is manually entered by volunteers. As a result, the completeness and accuracy of information such as weight, origin, or packaging details can vary significantly between products.
To ensure a basic level of reliability, the platform utilises barcode validation, structured data fields, and community-based moderation, allowing other users to flag incorrect or missing entries [7].

If a barcode is available, we first attempt to retrieve product data using the OpenFoodFacts API [7]. This database contains over three million food items and provides useful fields such as name, quantity, origin, packaging type, and CO_2 estimates.

When the API is unavailable or returns no results, we fall back to a locally stored CSV version of the database, which is converted to SQL for faster querying. However, the CSV is often outdated and incomplete (e.g., missing links to "Agribalyse" or CO_2 data), so additional enrichment is often required.

Step 2: Using Base Empreinte for Fresh Products

If the barcode is missing or unrecognized, a search is performed using the product name. The first name-based query targets the "Base Empreinte" database [42], specifically the section dedicated to fresh fruits and vegetables. This part of the database provides: seasonal availability (month-by-month), average CO_2 emissions, links to "Agribalyse" categories when available.

If a corresponding item is identified, we assume the product is fresh and use this data to estimate seasonality and emissions.

Step 3: Querying Agribalyse for Processed Products

If no entry is found in "Base Empreinte", we proceed with "Agribalyse" [6], developed by ADEME. This database provides life cycle assessment data for both raw and processed products, including greenhouse gas emissions and category classifications.

"Agribalyse" is especially useful when the product appears to be processed or when other sources lack sufficient data.

Product Name Matching Across Databases

We must align the product names with the database entries to extract meaningful data from these databases. This task is non-trivial due to inconsistent naming formats, abbreviations, or missing keywords. Therefore, we use a robust matching pipeline, detailed in the next section, that combines preprocessing techniques with similarity algorithms to identify the best match.

3.2 Text Preprocessing and Standardisation

Before applying similarity algorithms, we first need to standardise the product lines from receipts and the entries in the reference database. The aim is to eliminate unnecessary differences and retain only the essential parts for matching [43].

3.2.1 Text Cleaning

Product names on receipts are even more disorganised than those in databases. For this reason, after applying a standardisation process similar to the one used on the database, we add an extra filtering step to remove all stop words [43]. In our case, this also includes brand names (such as "Delhaize"), overly generic terms (like "bio"), or quantity indicators (such as "6pcs"). To achieve this, we assume that if a word is not found in the database's vocabulary, it likely doesn't provide helpful information for matching and can therefore be removed to avoid interfering with the algorithm. The final goal is to retain only the words that describe the product, such as "pomme", "compote", or "yaourt" [34].

These normalisation steps, when combined, allow us to standardise each word and reduce the distance between two similar items as much as possible, which helps the matching algorithm more easily find the correct correspondences [32].

3.2.2 Linguistic Normalization Techniques

Lemmatisation with NLP Libraries

This step aims to choose the most suitable natural language processing tool to convert our text into its base form. In our case, the "Agribalyse" database is mainly written in *French*. For this reason, we chose spaCy [27], a widely used and reliable tool supporting French.

The accuracy values shown in Table Table 3.1 are taken from the official Hugging Face platform [28] [29], [30], which provides benchmark results for each spaCy model using standard datasets such as UD_French-Sequoia.

Model	Accuracy	Loading Time	Execution Time
`fr_core_news_sm`	90.8%	∼1.5s	∼17.7s
`fr_core_news_md`	91.3%	∼2.9s	∼19.1s
`fr_core_news_lg`	91.4%	∼5.6s	∼23.9s

Table 3.1: Comparison of French spaCy Models for Lemmatisation

Execution times were measured locally using the list of 2,079 unique French nouns extracted from the "Agribalyse" database [6]. Each value represents the average of 10 test runs and includes both the model loading time and the time needed to process the complete list of words [41].

Among the three models, `fr_core_news_md` offers the best balance between accuracy and speed. While the `lg` model is slightly more accurate, it requires significantly more time to load and does not offer a noticeable improvement in processing speed or final results. Therefore, we selected the `md` model as the most suitable option for this project.

Stemming with NLTK Libraries

To perform stemming in French, we considered two algorithms based on the comparative study by Flores and Moreira [31]: the Paice/Husk stemmer and the Porter stemmer, the latter being available in Python through the `SnowballStemmer` class in the NLTK library [44].

Metric	Paice/Husk	Porter (Snowball)
Understemming Index (UI)	0.1650	0.2658
Overstemming Index (OI)	0.0001	0.0001
Stemming Weight (SW)	0.0007	0.0003
Relative Truncation Error (ERRT)	0.5131	0.6429

Comparison of Paice/Husk and Porter stemmers for French [31].

Explanation of metrics:

- **Understemming Index (UI)**: Measures how often the stemmer fails to group related words. Lower values are better.
- **Overstemming Index (OI)**: Shows how often unrelated words are incorrectly grouped. Lower is more precise.

- **Stemming Weight (SW)**: Ratio of OI to UI, showing the general behaviour of the stemmer (light or aggressive).
- **Relative Truncation Error (ERRT)**: Overall error compared to simple truncation. Closer to 0 means better quality.

According to the study, the **Paice/Husk** stemmer achieved better accuracy overall [31]. However, it is not included in standard NLP libraries, such as NLTK or spaCy, and is more complex to implement in practice.

In contrast, the **Porter (Snowball)** stemmer is widely used, easier to integrate, and well documented. Although slightly less accurate, it remains effective for practical applications, such as information retrieval. For this reason, we chose to use the Porter stemmer in our project.

Vocabulary Filtering

As explained in the introduction section 3.2, we now need to standardise the "Agribalyse" database [6] to make it easier to compare with the ticket data. The standardization process includes [43]: the preprocessing involves several steps, such as removing accents, capital letters, punctuation, and special characters; applying lemmatisation or stemming; and normalising units, such as converting "1L" to "1 litre." Once the standardisation is done, we build a dictionary containing every word found in this new version of the database. This dictionary will serve as a reference to identify important words during the ticket standardisation step.

3.3 Product Matching Pipeline

A critical part of this work is finding an effective way to match a product from a shopping receipt with its equivalent in an environmental database, such as "Agribalyse" [6] or ImpactCO_2 [42]. This match helps us gather additional information, such as the product's carbon footprint. The main difficulty is that product names often change from one store to another, depending on how the labels are written. Therefore, it is usually not possible to rely on exact name matching. For example, a line like `"Delhaize | Pommes | Royal gala | Emballé | 6pc 900 gr"` [45] should be matched with the standardised name `"Pomme, pulpe, cru"` in the "Agribalyse" database [6].

3.3.1 Candidate Refinement Techniques

Before selecting the best matching algorithm, an important step is to reduce the number of possible matches by filtering out irrelevant items from the "Agribalyse"

database [6].

To achieve this, we employ a filtering method that retains only the items that share the most meaningful tokens with the target string. This approach is inspired by the work of Zeakis [34], who proposed a token-based filtering technique using the concept of word importance.

Their method estimates the maximum similarity score between two sets of tokens based only on the words they have in common. This allows us to quickly remove unlikely matches before applying more complex similarity calculations.

In their experiments, this filtering technique reduced drastically the number of candidate matches, while maintaining a high level of matching accuracy.

3.3.2 Similarity Algorithms

In our system, product names extracted from shopping receipts are matched with reference items from the "Agribalyse" database [6]. For each product line, a similarity score is computed between the input name and all entries in the database using a predefined algorithm. The entry with the highest similarity score is selected as the best match. If this score exceeds a predefined threshold, the match is accepted and used to enrich the input with environmental data. Otherwise, the match is rejected, and the database cannot be used to complete the missing information for that product.

To perform reliable product matching, it is necessary to select the most appropriate similarity algorithm and determine an optimal threshold value. The threshold defines the minimum score required for a match to be accepted as valid. Choosing the right combination of algorithm and threshold is essential to maximise accuracy while limiting false positives.

Best Similarity Algorithm Evaluation

The following subsections introduce the different types of similarity algorithms that will be evaluated later in this work (see chapter 5). Each method has specific advantages and limitations depending on the structure and quality of the input data. These algorithms will be experimentally compared in a later chapter to identify the configuration that provides the best performance for our use case.

Token-Based Similarity Methods

We selected three classic token-based similarity methods: Cosine, Dice, and Jaccard. These algorithms represent each product name as a set of tokens and compute similarity using statistical formulas. This approach focuses on the presence of common words rather than exact string matches, making it more robust against variations in word order and length.

Our selection is supported by prior studies [32], [33], which show that these methods offer good performance for structured and clean data. However, in real-world applications such as matching names from grocery receipts, typographical errors, abbreviations, and inconsistent formatting are common. Under these noisy conditions, the effectiveness of Cosine(see subsection 2.2.4), Jaccard, and Dice decreases significantly [32].

Fuzzy Matching Techniques

To handle the limitations of token-based methods on noisy data, we incorporate fuzzy matching techniques [46]. These algorithms assess the degree of similarity between two strings, even when they are not the same. This flexibility is particularly valuable when dealing with errors or formatting inconsistencies, as minor spelling variations do not prevent valid matches.

While fuzzy methods offer improved recall, they are generally more computationally expensive and may introduce a higher number of false positives. Therefore, selecting an appropriate similarity threshold is crucial to strike a balance between accuracy and performance.

Character-Based Method: Jaro-Winkler

In addition to the token-based and fuzzy methods [46], we include Jaro-Winkler (see section 2.2.4), a character-based algorithm well-suited for short strings and typographical variations. It measures character-level similarity, giving more weight to common prefixes and suffixes. Its inclusion is justified by previous work [10], which shows its effectiveness in handling noisy text inputs.

Chapter 4

Carbon Footprint Estimation Pipeline

4.1 Overall Architecture

Once all available data has been collected and a preliminary carbon footprint has been found by using average values, we will refine this estimate by incorporating the complete dataset.

Climate impact data, provided by "Agribalyse" [6] and expressed in kilograms of CO2 equivalent per kilogram of product, are organised into seven categories, as presented in the state of the art section 2.2.1. These categories include global emissions (the total impact across all stages), agriculture (emissions from farming activities such as land use, fertilizers, and livestock), processing (industrial transformation of raw materials), packaging (production, use, and disposal of packaging), transport (logistics and freight emissions), retail (storage and display in stores), and consumption (emissions from cooking, refrigeration, and other consumer-level activities).

We aim to adjust the default values associated with each category using the detailed information collected during the data acquisition phase(see Figure 3.1), as outlined in(see section 2.1.2). For example, we refine the carbon footprint estimate in the agriculture category based on whether the product is in season or not [42]. For transport, we consider both the country of origin and the method of transportation used [12]. As for packaging, we take into account the type of materials and their respective weights [47].

Since "Agribalyse" provides average values for each category(see section 2.2.1), we

replace these with our computed estimates when sufficient data is available. If not, the original values are retained, which still offers a reliable approximation.

4.2 Fallback Estimation for Missing Data

In some cases, essential product information cannot be retrieved due to failed matching or incomplete input data. This section presents the different scenarios in which important data is missing, and explains how fallback strategies are used to estimate the carbon footprint despite these limitations.

4.2.1 Monetary-Based Emission Estimation

Using Price as a Proxy for Carbon Footprint

During the data collection phase, the matching process may fail (see section 3.1), or essential information, such as product weight, may be missing [7]. In such cases, a fallback method is used to estimate the carbon footprint using the product price as a proxy.

This method is inspired by current industry practices, particularly in the banking sector, where applications like those developed by BNP Paribas Fortis and Green-Got estimate emissions based on transaction data [48], [49].

The public monetary emission factor provided by BNP Paribas [37] is:

$$\text{CO38 - FOOD: } 556\,\text{g}\text{CO}_2\text{eq}e$$

For instance, in May 2025, a “Blueberry Muffin Bar” priced at €3.29 at Collect & Go [50], with no "Agribalyse" data available, is estimated as:

$$0.556 \times 3.29 = 1.83\,\text{kg CO}_2\text{eq}$$

4.3 Agricultural Emissions Estimation

To estimate greenhouse gas emissions from agriculture, it is essential to consider whether fruits and vegetables are in season or not [42]. Seasonality has a significant impact on the carbon footprint of products. For example, a tomato cultivated locally in a cold country, such as Belgium, during winter generally has a significantly higher carbon footprint than a tomato grown in a warmer country like Spain during the same period.

According to a study by Agrobio Bretagne [51], the carbon footprint of off-season tomatoes grown in heated greenhouses in France can reach up to 2.2 kgCO2e/kg, whereas in-season, field-grown tomatoes emit only about 0.3 kgCO2e/kg. This represents a more than sevenfold increase. This comparison illustrates how seasonality can significantly impact the environmental footprint of agricultural products.

4.3.1 Algorithm Description

This section explains the algorithm used to calculate agricultural emissions with a focus on the seasonality of fresh products. Due to the lack of detailed data on the carbon cost of heating greenhouses and product storage, the algorithm is currently limited to the values provided by the "Agribalyse" database(see section 2.2.1). These values include seasonal variations only for strawberries and tomatoes [22]. For all other products, default values from "Agribalyse" are used.

Decision Logic

The main decision logic used to estimate agricultural emissions is illustrated in Figure 4.1. The algorithm begins by checking whether the product belongs to a category for which specific carbon emission data is available for both in-season and out-of-season periods, such as strawberries or tomatoes. If no seasonal emission data exists for the product, the algorithm defaults to the average values provided by the "Agribalyse" database.

If seasonal data is available, the next step is to evaluate the country of origin. When the product originates from a warm country, such as Spain, it is assumed to have been grown in open-field conditions and therefore considered in-season, as supported by [52]. Conversely, if the origin is a cold country, such as France, the system checks whether information about seasonal availability is available. If such data exists, the purchase date is compared to the seasonal calendar to determine whether the product is in or out of season. Based on this, the corresponding in-season or out-of-season emission factor from "Agribalyse" is applied. If no seasonal calendar is available, the default value is used.

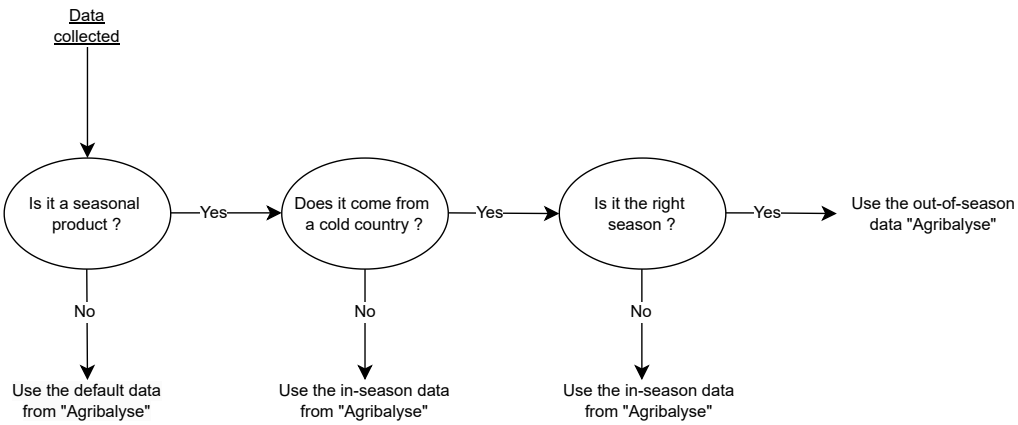

Figure 4.1: Main steps of the algorithm for estimating agricultural emissions.

4.3.2 Seasonality Effects

Heated Greenhouse Estimation by Country

To assess whether a country is likely to grow certain crops in heated greenhouses, we use the methodology proposed by Perpiña Castillo [53], which estimates energy consumption in greenhouses based on crop types and climatic zones.

The table below presents average energy use (in kWh/m²/year) for selected crops across various countries:

Country	Tomato	Cucumber	Lettuce	Eggplant	Strawberry
France	**172**	194	61	245	**194**
Germany	**276**	303	82	341	**303**
Italy	**117**	136	35	164	**136**
Netherlands	**234**	260	78	300	**260**
Spain	**72**	93	0	123	**93**
Sweden	**530**	564	267	599	**564**

Table 4.1: Greenhouse energy consumption per crop and country

According to the French Ministry of Agriculture [54], greenhouses typically consume between 200 and 500 kWh/m²/year. Based on this, we apply a threshold of 200 kWh/m²/year to determine whether a product is considered off-season and grown in a heated greenhouse. If the value is below this threshold, the product is considered seasonal.

EU vs Non-EU Production Assumptions

If the product originates from outside the European Union, we assume it was grown in favourable open-field conditions and not in a heated greenhouse [52]. This assumption reflects the typical trade patterns where EU countries import fresh produce from regions with warm climates.

Using Seasonal Calendar Data

To determine if a product is in season at the time of purchase, we use monthly availability data from the "Base Empreinte" database(see section 2.2.1). If the purchase date matches a seasonal month for that product, the in-season emission factor from "Agribalyse" is used. Otherwise, we apply the out-of-season value from "Agribalyse"(see section 2.2.1).

Currently, this method is implemented only for strawberries and tomatoes, due to data availability(see subsection 4.3.1). However, the logic is generic and can be extended to other products once more seasonal data becomes accessible.

4.4 Transport Emissions Estimation

In this section, we outline the method for estimating the share of the carbon footprint attributable to transportation. As with other stages of the product life cycle, "Agribalyse" provides a default estimate of transport-related emissions. However, this value is generic and does not account for product-specific factors(see section 2.2.1). Our objective is to refine this estimate using available contextual data.

To achieve this, we first determine the geographic coordinates (latitude and longitude) of the product's country of origin and then compute the most probable transport route to the point of sale [19], [55]. Based on this route, we identify the likely mode of transport used, which allows for a more accurate calculation of CO2eq emissions associated with the journey. Additionally, we assess whether the product was transported under ambient or refrigerated conditions [8], as refrigeration significantly affects emissions.

Due to the complexity and variability of supply chains for processed products, we limit this refinement to unprocessed fresh products, such as fruits and vegetables. For all other items, we rely on the default values provided by "Agribalyse", which offer a reasonable estimate for products that have undergone transformation.

4.4.1 Algorithm Description

This section describes the algorithm used to calculate the carbon footprint associated with transportation. The diagram below (see Figure 4.2) illustrates a state machine that represents the decision-making process implemented in the program to determine which data should be used for an accurate estimate of the carbon footprint.

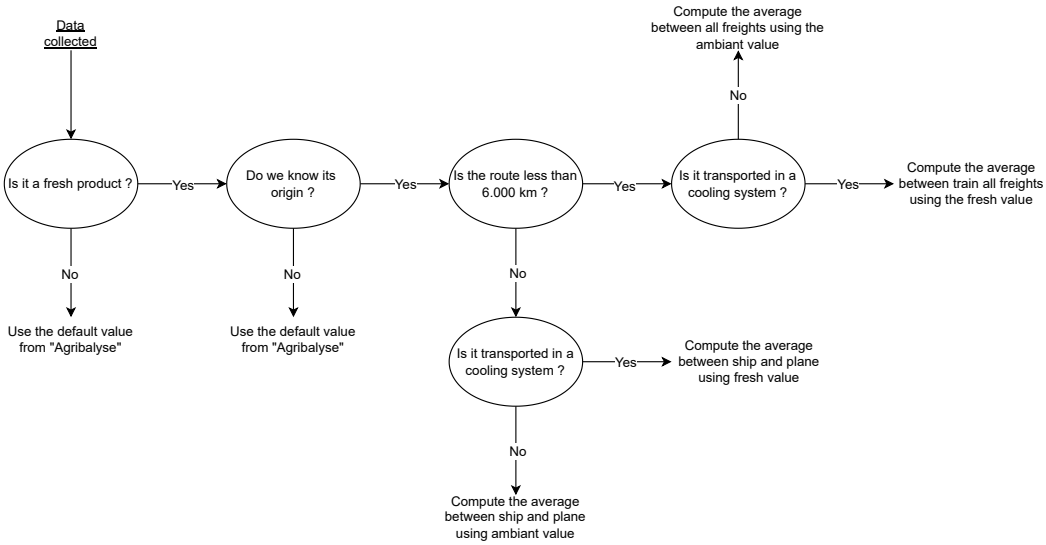

Figure 4.2: State machine representing the decision process for selecting transportation data

As stated in the introduction(see section 4.4), this method applies exclusively to fresh products such as fruits and vegetables. To determine whether a product meets these criteria, we refer to the "Base Empreinte" database(see section 2.2.1), which has a section that includes only fruits and vegetables.

We first check whether information about the country of origin is available. If not, we apply the default transportation emission values from the "Agribalyse" database(see section 2.2.1).

If the country of origin is known, we assess whether it is accessible by truck using the OpenRouteService API [35]. As explained in section 2.2.5, if the calculated route exceeds 6000 km, the destination is considered out of range for road transport. In such cases, transport emissions are estimated as the average between maritime and air freight emissions.

Suppose the country of origin is located in Europe. In that case, the next step is to assess the product's transport conditions specifically, whether it requires

refrigeration (as with fresh products) or can be transported at ambient temperature. For fresh products, the algorithm applies the average emissions associated with boat and truck transport under refrigerated conditions. In contrast, for products that can be transported without refrigeration, the average emissions from train and truck transport under ambient conditions are used.

Finally, in all cases, if none of the required information is available (product type or origin), we default to the generic transportation emission factor of "Agribalyse".

4.4.2 Transport Mode Selection

As described in the algorithm Figure 4.2, the choice of transport mode primarily depends on the distance between the country of origin and the destination, as well as the perishability of the product [56]. However, due to the lack of precise and reliable data indicating the transport mode by product type and country, we rely on global averages provided by Poore and Nemecek (2018) [57]. Their study, cited by Our World in Data, presents the global distribution of food miles by transport mode, which we use as an approximation for our model.

The graph below illustrates the global food miles, measured in tonne-kilometres per year.

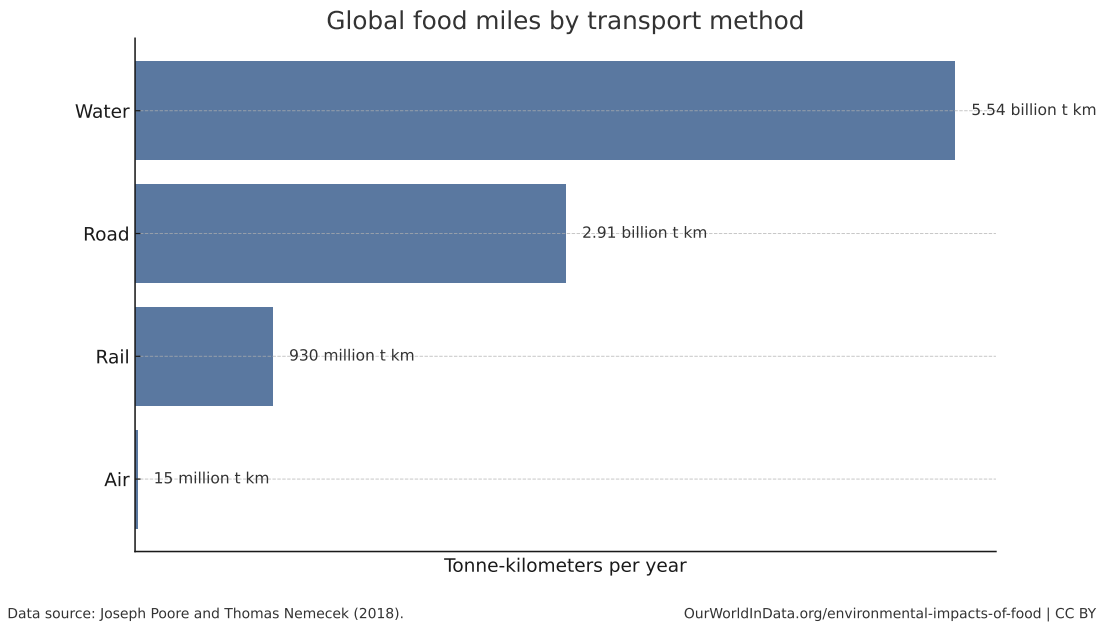

Figure 4.3: Global distribution of food miles by transport mode, measured in tonne-kilometres per year.

According to the data, maritime shipping accounts for the highest share of food transport, reaching 5.54 billion tonne-kilometres per year. This corresponds to approximately 64.4% of the total food miles. It is followed by road transport (27.9%), rail transport (7.0%), and air transport (0.7%). These proportions are used to estimate the average contribution of each transport mode such as road, sea, rail, or air in the overall distribution of food miles. This distribution is then applied in our final calculation to approximate the share of emissions associated with each mode of transport when detailed routing information is unavailable.

However, since we use OpenRouteService (see section 2.2.5) to compute distances for road and rail transport, the maximum calculable distance is limited to 6,000 km. Beyond this limit, the API returns an error. As such, this value will serve as the upper threshold for terrestrial transport, or as an indicator that no terrestrial route is available when the API fails to generate a result. This threshold is generous, considering Eurostat reports an average distance of 611.3 km for freight transported by truck [58].

In such cases, we exclude road and rail transport from the proportional distribution and reassign the percentages accordingly, resulting in 90% for maritime shipping and 10% for air transport(see Figure 4.3).

4.4.3 Determining Product Origin

To calculate the distance traveled by freight transport, it is first necessary to estimate the geographic coordinates of the product's origin and destination as precisely as possible. This estimation is based on the data collected during the information gathering phase (see section 3.1). The country of origin was identified for many products using OpenFoodFacts (see section 2.2.1).

A straightforward approach would be to use the geographic centroid of the country as a representative location. However, this method can lead to complications when calculating the shortest route, as the centroid may not correspond to a habitable or accessible area, possibly causing errors in routing calculations [35]. This issue was encountered with countries such as China and France.

To avoid such errors, we opted instead to use the geographic coordinates of each country's capital city. This approach is commonly adopted in economic studies, including the CEPII GeoDist database [59], where distances between countries are computed based on the coordinates of their respective capitals.

4.4.4 Emission Factors by Modes

To estimate the carbon footprint associated with transportation, it is essential to determine the distance travelled. In this thesis, we consider the four main modes of freight(see Figure 4.3) transport and identify, for each, the most accurate method to estimate the most likely route taken to deliver food products to the store. This allows us to calculate the total distance travelled.

First, we explain how the origin and destination points of each route are estimated. Then, we describe in detail, for each mode of transport, the algorithm used and the reasoning behind our approach.

Truck Transport Estimation

To estimate the delivery route of trucks transporting fresh products to retail stores, we calculate the shortest path accessible by trucks between the origin and the destination. This calculation is performed using the OpenRouteService API [35], which offers routing data optimised for different transportation modes, including heavy goods vehicles.

The API accounts for actual road infrastructure and routing constraints. It is based on OpenStreetMap data and provides distance and duration estimates with high spatial accuracy.

However, as indicated in Figure 4.4.2, the API imposes a limit of 6,000 km. Beyond this threshold, it returns an error. Consequently, this value serves as the upper limit for terrestrial transport or as an indicator that no terrestrial route is available if the API fails to produce a result. This threshold is relatively generous, given that Eurostat reports an average distance of 611.3 km for freight transported by road [58].

Once the most likely truck route distance is determined, the next step is to select the appropriate CO2 impact factor for the truck, based on its refrigeration mode. The available options are presented in Table 4.2 and dive by the "Base Empreinte" database in the transport section [8].

Truck Type	Temperature Condition	Emission Factor
Ambient truck	Ambient temperature	0.089 kg CO2 eq./t·km
Refrigerated truck	Cool temperature	0.147 kg CO2 eq./t·km

Table 4.2: Available emission factors based on truck refrigeration mode

To determine which emission factor to use, we refer to data from the La Bella Frutta website [60], which provides a detailed table of storage conditions for fruits

and vegetables. This table indicates the recommended storage durations for each product type, specifying whether it requires ambient or refrigerated storage. It helps us estimate storage needs more accurately, and thus, the related carbon footprint.

When the table indicates "Not recommended" for ambient storage, refrigeration is necessary, and the emission factor for refrigerated trucks is used. Otherwise, if the product can be stored for a few days under ambient conditions, the ambient temperature emission factor is applied.

With both the distance travelled and the selected emission factor known, the total emissions are calculated by multiplying the distance (in kilometres) by the emission factor, and dividing by 1,000 to express the result in kilograms of CO2 per kilogram of product transported [16].

The example below illustrates a route and calculation based on the following input parameters:

- Origin: France
- Destination: Estonia

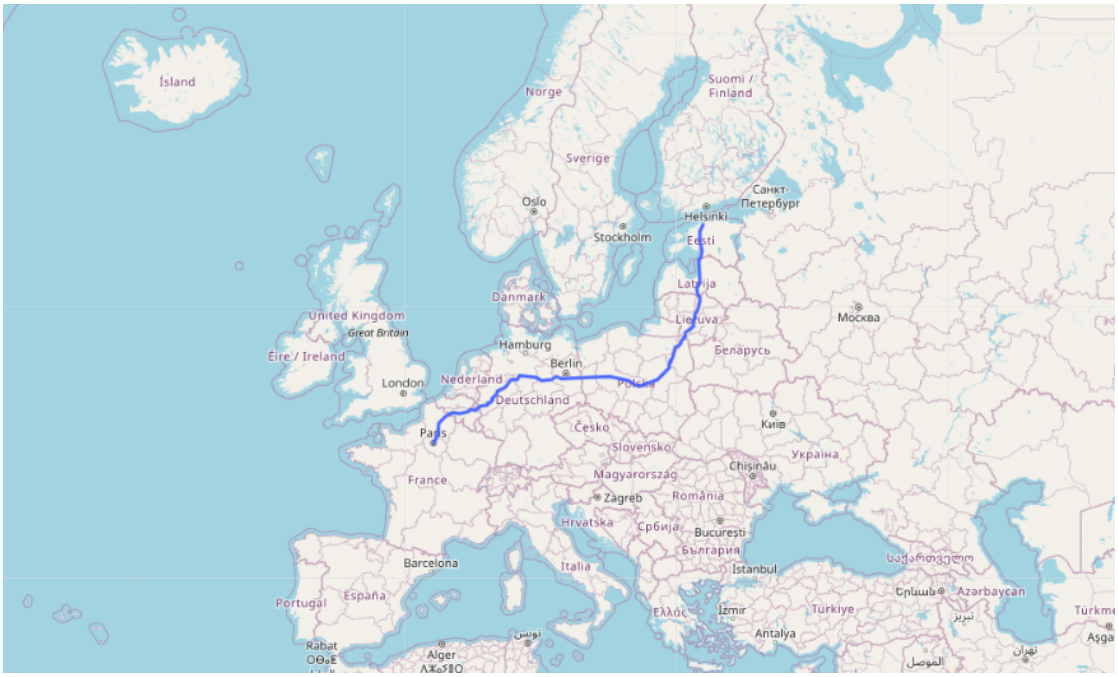

Figure 4.4: Example of truck route from Paris to Tallinn

As shown in Figure 4.4, the blue line represents the path taken by the truck from

Paris, France, to Tallinn, Estonia, covering a distance of 2,549 km.

Rail Transport Estimation

Estimating emissions for train transport is more complex due to the absence of a freely accessible program capable of calculating the shortest route between two railway stations. However, according to Carbon4 [55], in 90% of cases, the distance travelled by road is within 90% to 110% of the equivalent rail distance, with observed deviations ranging from -19% to +34%. On average, the distance by road is less than 1% shorter than by rail, without weighting by population or passenger numbers.

Based on this observation, we consider the road transport distance as a valid proxy for rail transport distance, introducing a maximum average error margin of approximately 1%.

For the emission factor, we use the value provided by the "Base Empreinte" dataset [8], specifically for the category “Transport de marchandises en train électrique” in Europe (excluding Switzerland). This dataset reports a mean value of 3.67E-02 kg CO2 eq./tonne · km, representing the average emissions for freight transported by electric train across Europe.

To estimate the carbon footprint associated with transporting a product by train, we multiply this emission factor by the approximated travel distance.

Maritime Transport Estimation

The logic behind estimating boat freight follows the same principles as those for trucks and trains. First, the transport distance must be calculated. For this purpose, we use the open-source searoute Python package [36]. This tool computes the shortest maritime route between two geographic coordinates using a global naval network developed by Eurostat. It is based on Dijkstra’s algorithm, which identifies the most efficient path along predefined maritime corridors.

Unlike commercial APIs, this package does not consider real-time vessel data or constraints such as emission control zones or piracy areas. Nevertheless, it provides a reliable estimate of the physical route length based on standardised navigation graphs.

Once the distance is calculated, it is multiplied by the emission factor associated with maritime freight [16]. For our analysis, we use values from the "Base Empreinte" database [8]. This category is divided into two values depending on the transport conditions:

Ship Type	Temperature Condition	Emission Factor
Ambient ship	Ambient temperature	0.0102 kg CO2 eq./t·km
Refrigerated ship	Controlled temperature	0.0227 kg CO2 eq./t·km

Table 4.3: Emission factors for maritime freight by temperature condition

To determine which emission factor to apply, we refer to the same logic presented in the truck transport Table 4.2, which explains how to infer temperature conditions from product data.

Once the distance and the appropriate emission factor are identified, their product yields the estimated carbon footprint associated with maritime transportation.

The example below illustrates a route and calculation performed using the following input:

- Origin: Belgium
- Destination: China

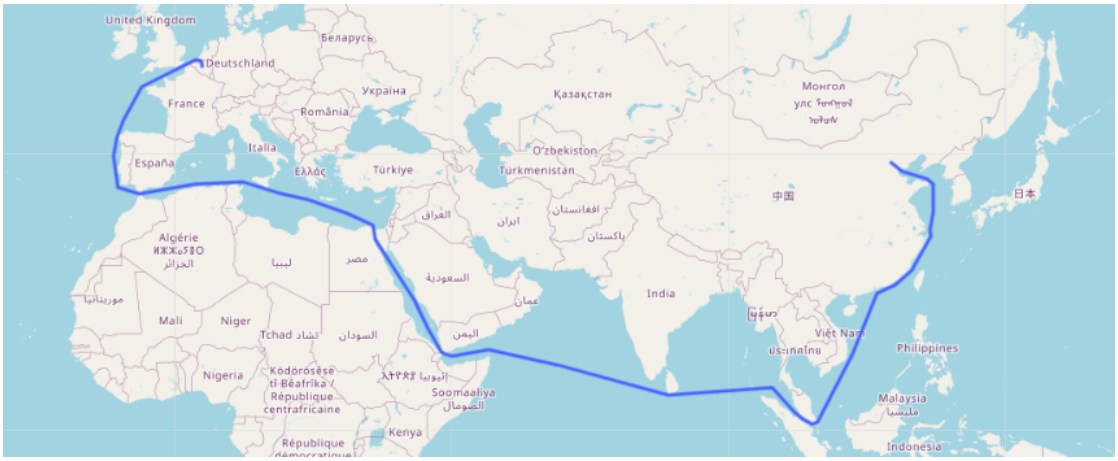

Figure 4.5: Example of maritime route from Belgium to China computed with searoute

As shown in Figure 4.5, the blue line represents the maritime route from Brussels (Belgium) to Beijing (China), covering a total distance of 20,690.5 km.

Air Freight Estimation

To estimate the CO2 emissions from air freight, we first calculate the great-circle distance between the origin and destination points using the Haversine formula.

This formula, based on spherical trigonometry, provides the shortest distance between two points on a sphere using their geographic coordinates. We use the implementation available in the Python package haversine [61]. This method is also employed by established emissions calculators such as MyClimate [62], which use great-circle distances as a basis for estimating flight-related CO2 emissions.

Once the distance is calculated, we determine the CO2 emission factor associated with the flight. According to the BaseEmpreinte database(see section 2.2.1), emissions per tonne-kilometer vary depending on aircraft size, the presence of contrails, and flight distance. Since we cannot determine the specific aircraft type for each product, we rely on average values provided by EcoInvent via the BaseEmpreinte platform [8].

The following table shows the emission factors by flight distance:

Category	Distance Range	Emission Factor
Very short	$\leq$ 500 km	1.41 kg CO2 eq./t·km
Short	$>$ 500 km and $\leq$ 1000 km	1.09 kg CO2 eq./t·km
Medium	$>$ 1000 km and $\leq$ 3500 km	0.934 kg CO2 eq./t·km
Long	$>$ 3500 km	0.803 kg CO2 eq./t·km

Table 4.4: Emission factors for air freight by distance category

As shown in Table 4.4, shorter flights emit more CO2 per kilometer. This is mainly due to the relatively higher fuel consumption during take-off and landing phases on short-haul flights. Moreover, aircraft used for short distances are typically less fuel-efficient per tonne-kilometer than those used for long-haul flights [63].

Finally, the total emissions are obtained by multiplying the distance (in kilometers) by the corresponding emission factor, and dividing by 1000 to express the result in kilograms of CO2 per kilogram of product transported [16].

This method provides a realistic estimate of air freight emissions based on the country of origin of each product.

4.5 Packaging Emissions Estimation

We now arrive at the final stage of our carbon footprint estimation: calculating the greenhouse gas (GHG) emissions associated with product packaging. As with previous steps, the precision of this calculation depends largely on the availability of information collected during the data gathering phase (see section 3.1).

In this section, we present the algorithm used to estimate the packaging-related carbon footprint as accurately as possible. We detail the different scenarios and explain the computation method applied depending on the type and completeness of the available data.

4.5.1 Algorithm Description

Here we will describe the decision tree used to determine how the carbon footprint of packaging is estimated, depending on the available data, Figure 4.6.

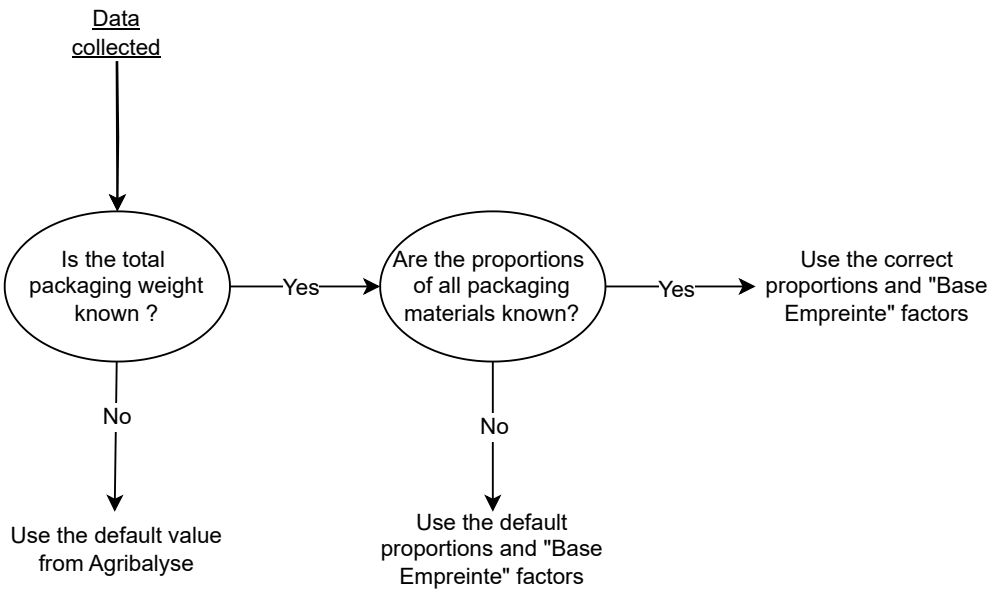

Figure 4.6: Decision tree used to estimate packaging emissions based on available data

In cases where no specific information is available on the packaging, the algorithm defaults to using the average value provided by the "Agribalyse" database (see section 2.2.1). However, when some packaging information is available (see section 3.1), the estimation process adapts based on the level of detail provided.

If the total packaging weight is known but the material composition is not, the system applies default material proportions along with their associated emission factors. In contrast, when both the total weight and the share of each material type are known, these exact values are used to calculate a more precise estimate.

In this case, emission factors from "Base Empreinte" are applied (see section 2.2.1) to improve the accuracy of the result.

4.5.2 Unknown Composition

Assumptions Based on food Sector Averages

When the exact material composition of a product's packaging is unknown but its weight is available, we estimate the carbon footprint using a weighted average. This average is based on typical material usage proportions in the food sector [16].

The reference values are derived from AgriRéseau [64], which provides sector-specific data for the Quebec market. While global datasets often include only plastic and cardboard, the Quebec data offers a more comprehensive breakdown of four commonly used materials: paper/cardboard, plastic, metal, and glass.

Although CNA-Alimentation [47] offers emission factors for packaging in European household products, it was not selected here. The CNA dataset is not specific to food products, unlike the AgriRéseau reference, which more closely matches the food-sector packaging context studied in this thesis. Therefore, we consider the Quebec data a more appropriate approximation for European food packaging.

Emission Factor Calculation

The proportions and corresponding emission factors used are shown in Table 4.5.

Material	Proportion (%)	Emission Factor
Paper or cardboard	41	1.006 kg CO2 eq./t·km
Plastic	26	2.380 kg CO2 eq./t·km
Metal	10	9.830 kg CO2 eq./t·km
Glass	23	0.810 kg CO2 eq./t·km

Table 4.5: Emission factors by material type for unknown packaging

The average emission factor for packaging with unknown material composition is calculated by multiplying the CO_2 emission factor of each packaging material by its average proportion, as defined in Table 4.5. This results in a composite emission factor representing a typical packaging mix. It can then be used to estimate the packaging emissions when the total packaging weight is known but its material breakdown is not. Here is the calculation

$$0.41 \times 1.006 + 0.26 \times 2.38 + 0.10 \times 9.83 + 0.23 \times 0.810 = 2.2 \text{ kg CO2 eq./kg}$$

4.5.3 Known Composition

In some cases, OpenFoodFacts(see section 2.2.1) provides detailed information on the proportion of each material used in a product's packaging. When such data is available, the calculation of the packaging's carbon footprint becomes straightforward.

We simply use the material proportions provided by OpenFoodFacts and multiply each by its corresponding emission factor [16], as presented in Table 4.5. The result is the estimated carbon footprint of the product's packaging, expressed in kg CO2 eq. per kilogram of packaging.

Chapter 5

Evaluation

In this chapter, we evaluate the performance of the matching algorithms described earlier (see chapter 3). Our goal is to determine which combination of preprocessing techniques (such as stemming, filtering, and normalisation) and similarity measures gives the highest match rate.

The evaluation is divided into several parts. First, we compare the mean accuracy of each algorithm using different preprocessing settings. Next, we analyse the execution time to see which method is the fastest. We then discuss how the optimal similarity threshold was chosen by balancing precision and recall to select the best algorithm for our case.

These tests help us understand which matching approaches are best suited to our task and guide the selection of the most effective parameters to maximise both reliability and match rate.

5.1 Threats to Validity

Despite promising results in controlled settings, several factors limit the external validity of this work.

First, the system has not been tested on real-world grocery receipts. Most physical receipts do not include complete or readable barcodes, which requires manually identifying the correct barcode for each product. In many cases, stores do not publish product barcodes on their websites, further complicating the process. Building a comprehensive and representative dataset of real receipts would involve significant time and manual effort, which was not compatible with the time constraints of this Thesis. Therefore, a large-scale evaluation on real receipts has been left for future

work.

Second, and more critically, the system could not be validated against verified real-world carbon footprint values. To the best of our knowledge, there is currently no publicly available dataset providing ground-truth emissions data for individual products, considering factors such as origin, transportation, farming method, and packaging. This lack of reliable reference values prevents any direct assessment of the estimation accuracy. Consequently, the system's precision remains unquantified, which weakens its scientific validation.

These limitations imply that the current evaluation demonstrates only internal consistency and technical feasibility under controlled, synthetic input conditions. While the system serves as a proof of concept, further research is necessary to validate its robustness under operational conditions.

Future work should focus on constructing a validated receipt dataset with real product-to-barcode mappings and finding or creating trusted carbon footprint data for everyday retail products. Having this kind of reference is essential for checking the accuracy of the system and for going beyond just theoretical results.

5.2 Experimental Results

This section presents the evaluation of our system through a series of experiments. We begin by justifying the sample size used for validation, then assess the accuracy and performance of different matching algorithms and preprocessing strategies. We also analyse execution times and finally examine the effect of keyword-based score adjustments on overall matching performance.

Sample Size Justification

To evaluate the robustness of our matching system, we cannot test all possible cases to precisely assess the algorithm's matching rate. Since the number of potential product names and descriptions is virtually infinite, including variations in names, barcodes, and formats, it is only feasible to evaluate the system on a representative sample.

For this, we manually tested its accuracy on a set of 210 items extracted from receipts issued by various vendors. The goal was to obtain a sample that was as representative as possible of real-world cases.

To know if this sample is enough to be representative of the reality based on an almost infinite number of possible cases, we use the margin of error [65].

The margin of error helps evaluate the size of our sample because it indicates how close our measured performance is likely to be to the actual performance of the system on the full population. It quantifies the uncertainty due to sampling, and allows us to judge whether the number of tested examples is statistically sufficient to support reliable conclusions [65].

In particular, a smaller margin of error indicates that the sample size is sufficiently large to yield a stable estimate. In contrast, a larger margin reflects greater variability and reduced confidence in the results. To properly assess the reliability of our findings, we calculate a confidence interval. Given that matching errors are costly and can compromise the relevance of our estimates, we adopt a 95% confidence level, which provides a reasonable balance between precision and caution [65].

Note on Statistical Reliability

According to Bartlett et al. [66], a margin of error below ±5 percentage points at the 95% confidence level is generally considered sufficient to support conclusions about survey-based research reliability.

However, since the sample was not selected through a truly random process, we cannot directly rely on the observed accuracy to calculate a precise confidence interval. Following standard statistical practice, in the absence of a randomised sample, we report a conservative estimate of the margin of error using the worst-case scenario assuming a proportion $p = 0.5$, which maximises uncertainty [67].

Using the standard formula for margin of error at a 95% confidence level:

$$\text{MoE} = z \times \sqrt{\frac{p(1-p)}{n}}$$

Where:

- $z = 1.96$ (for 95% confidence),
- $p = 0.5$ (worst-case scenario),
- $n = 210$ (sample size),

We compute:

$$\text{Margin of error} = 1.96 \times \sqrt{\frac{0.25}{210}} \approx 1.96 \times 0.0345 \approx 0.0676$$

This results in a conservative margin of error of $\approx 6.8\%$.

Although this margin of error slightly exceeds the commonly referenced five percentage point margin in the literature, it remains within an acceptable range to support preliminary yet meaningful conclusions. As acknowledged in several professional survey methodology guides, such as those published by LimeSurvey [68], Zoho Survey [69], and SurveyLab [70], a margin of error between 4% and 8% at the 95% confidence level is generally considered acceptable in exploratory or early-stage research.

Nevertheless, to validate these findings with greater statistical rigour, we recommend repeating the evaluation with a sample of 384 items, which would reduce the margin of error to 5%, as suggested by Bartlett et al. [66].

5.2.1 Matching Accuracy

After conducting our sample test, we manually matched each product name from the test set to its corresponding entry in the "Agribalyse" database [6]. The evaluation is binary: the algorithm receives 1 point for a correct match and 0 for an incorrect one.

The chart in Figure 5.1 compares the performance of each algorithm across different preprocessing strategies: lemmatisation, stemming, filtering, and a baseline without any preprocessing.

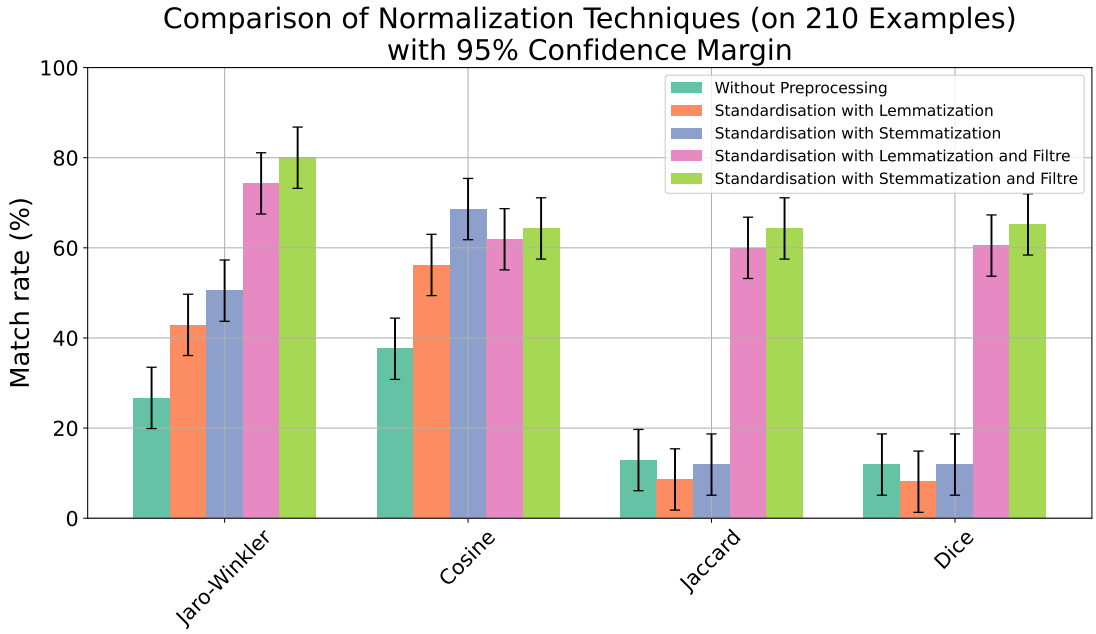

Figure 5.1: Comparison of matching accuracy for different algorithms using various preprocessing configurations

Observations

Key Result

Jaro-Winkler achieves the best matching accuracy in our tests, reaching 80.00% ± 6.8% with stemmatization and filtering. It is the most reliable method for noisy, short product names.

From our experiments, the Jaro-Winkler and Cosine similarity algorithms clearly stand out as the most effective under multiple preprocessing conditions. Their accuracy improves significantly when filtering is applied, especially when combined with standardisation through stemming.

With the configuration using stemmatization and filtering, Jaro-Winkler achieves the highest matching accuracy at 80.00% ± 6.8. Dice and Cosine follow closely with 65.24% and a margin of error of ± 6.8 and 64.29% ± 6.8, respectively. Jaccard also reaches 64.29% ± 6.8 in this setup.

These results demonstrate that the Jaro-Winkler method works particularly well when the number of comparisons is limited, likely because it handles spelling variations and minor formatting issues more effectively than other methods.

The Impact of Filtering on Simpler Methods

Without filtering, Jaccard and Dice show limited performance (below 13%). However, their accuracy improves significantly when filtering and stemming are applied, reaching over 64%. This sharp improvement highlights that both methods are susceptible to noise in the data and benefit strongly from candidate preselection.

Both Jaccard and Dice are token-based. They compute similarity by comparing shared words between strings section 2.2.4. However, in their raw form, product names often include additional elements such as store names, quantities, or inconsistent phrasing, which introduce noise and lower match accuracy.

Applying standardisation section 3.2 and filtering subsection 3.3.1 helps clean the input and reduce the number of irrelevant database entries. Only the most relevant candidates, those sharing significant tokens, are retained. In this smaller, cleaner set, Jaccard and Dice are much more effective.

Strengths of Jaro-Winkler and Cosine

Jaro-Winkler section 2.2.4 is based on character-level similarity. It places more weight on matching prefixes, making it ideal for shorter or slightly messy strings. For example, "Pomme Emballée 6pcs" and "Pommes pulp cru" share enough characters, especially at the beginning, for Jaro-Winkler to consider them similar, even if their tokens differ.

Cosine similarity, on the other hand, is more effective when product names are cleanly tokenised and well-structured. It relies on vector representations of words, making it better suited for longer or more detailed strings.

In practice, Jaro-Winkler is well-suited for final matching steps when the candidate list is small. Cosine remains useful earlier in the process as a pre-filtering tool, where its broader recall helps narrow down relevant options(see subsection 3.3.1).

5.2.2 Execution Time Analysis

To select the most suitable algorithm, it is essential not only to consider accuracy but also execution time. We evaluated execution performance using the same dataset as for the accuracy analysis. For each algorithm, we measured the execution time per item and calculated the average, minimum, maximum, and standard deviation [41].

The results, as shown in Figure 5.2, indicate that the combination of stemming and filtering yields the fastest execution time, with an average of 22.5 ms. This is significantly lower than using stemming alone (140.9 ms) or no preprocessing

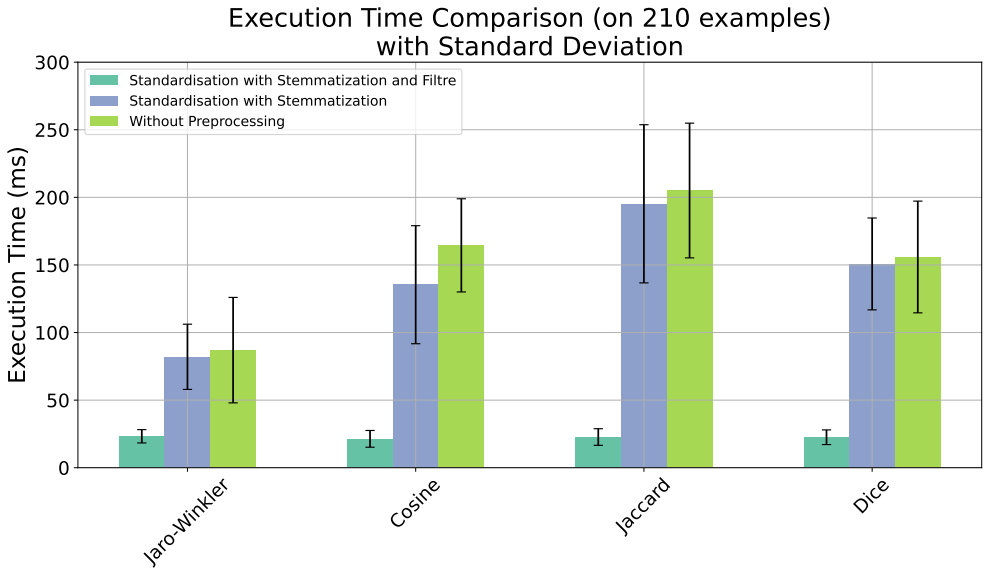

Figure 5.2: Comparison of execution times in milliseconds for four string similarity algorithms under different text preprocessing scenarios.

(153 ms). This confirms that filtering helps reduce computation time by narrowing down the number of comparisons needed.

We also assessed the consistency of execution times. Standard deviation provides insight into the stability of performance across different inputs. All methods exhibited relatively stable timing, with standard deviations ranging from 21% (with filtering) to 28% (without filtering) of the average. This level of variability is expected when working with heterogeneous data. It does not indicate a lack of stability in the algorithm, but rather reflects the diversity of receipt inputs. Some are short and clean, others are long, noisy, or inconsistently formatted. Differences in string length, token complexity, and the number of candidates being compared all contribute to fluctuations in execution time across samples.

Notably, when no filtering is applied, the performance gap between methods becomes more apparent. In that case, Jaro-Winkler is up to twice as fast as the other methods. However, once filtering is introduced, the number of comparisons is significantly reduced, and execution times across all algorithms converge to similar values.

5.2.3 Impact of Keyword Scoring

Keyword-Based Score Adjustment

To improve the match rate, we identified specific descriptive keywords that could influence the similarity score. We selected the three most common adjective-like terms. We evaluated the impact of assigning a bonus or penalty to the score when one of these words appears in an "Agribalyse" item.

The selection of these keywords was made arbitrarily to strike a balance between test execution time and the frequency with which each word appears in the dataset. The three most representative words chosen for this experiment are "cru" (occurs 211 times in the dataset), "sec"(77 times), and *"surgelé"*(71 times).

In this study, we limit our tests to these three descriptors to observe the general influence of applying a bonus or a malus to key terms. However, if the objectives were to find the optimal parameter configuration, this process could be extended to a broader set of keywords.

Sequential Parameter Testing

Methodological Perspective

While the current experiment uses an arbitrary selection of three keywords, a more rigorous approach would involve training on a labelled dataset to learn optimal weights, followed by validation on a separate test set. This would improve generalisation and reduce overfitting.

In this section, we limit ourselves to testing the effect of each parameter separately. Testing the parameters sequentially yields linear complexity, whereas testing all possible combinations would result in exponential complexity.

Although more advanced methods such as random search, coordinate descent, or Bayesian optimisation can significantly reduce the number of iterations needed to find near-optimal parameters [71], the goal of this experiment is not to optimise the configuration but to illustrate the general influence of keyword bonuses and penalties on the match rate. For this reason, we opted for a sequential exploration of each parameter.

Effect of Specific Term Weights

We tested different values for these bonuses and penalties using the same test set database as the accuracy test section to find the best combination that improves

the match rate. The lane represents the observed value, and the colored area represents the margin of error of the test.

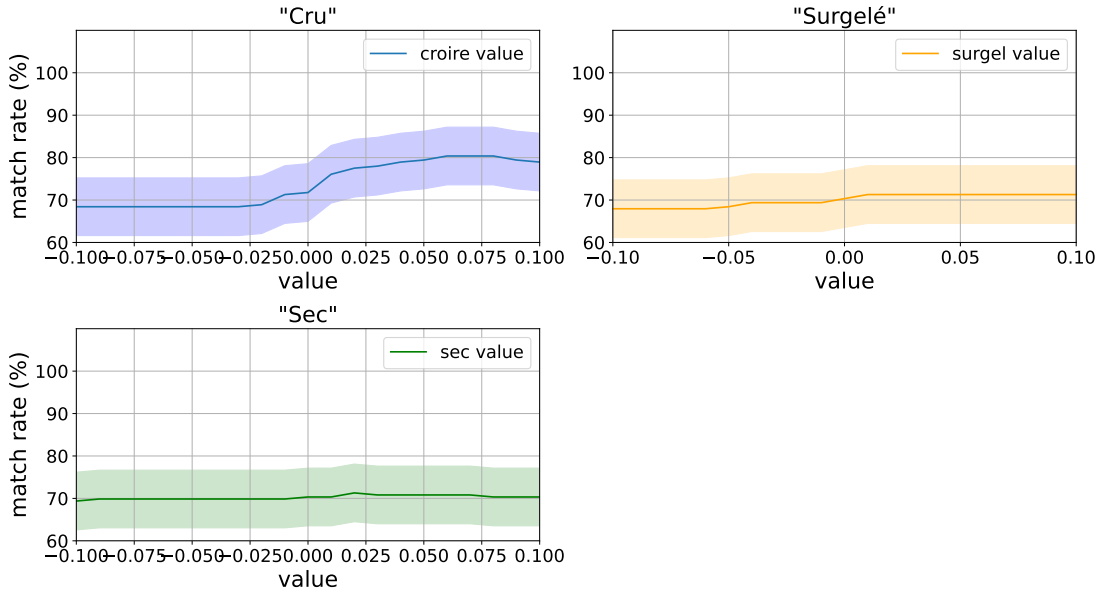

Figure 5.3: Effect of keyword bonuses and penalties on the match rate

Based on the results shown in the graph Figure 5.3, we identified the parameter values that yield the highest match rate. The best values found were: bonus("cru") = [0.06,0.08], penalty("surgelé") = [0.01,0.1], and bonus("sec") = 0.02.

However, it is essential to consider the margin of error. In our case, the error margin is relatively large compared to the gains obtained for the words "surgelé" and "sec". This suggests that the observed improvement may have occurred by chance, and the uncertainty is too high to conclude that this keyword had a meaningful positive effect confidently.

In contrast, the word "cru" shows an improvement of more than 6.8%, which exceeds the margin of error. This indicates a reliable and statistically significant contribution to the algorithm's performance. This strong result may be explained by the high frequency of the word in the dataset (211 occurrences), compared to "surgelé" (71 occurrences), which likely increases its impact on the final matching score..

Nevertheless, this result suggests that "sec" or "surgelé" could have a complementary effect when combined with other keywords. Exploring such interactions could reveal stronger correlations and potentially yield better results.

In conclusion, although isolated tuning yields mixed outcomes, especially under statistical uncertainty, combining keyword scores in a more integrated way appears to be a promising direction for future investigation.

5.3 Threshold Calibration

Once the Jaro-Winkler [10] algorithm was selected as the most effective method for matching product names section 5.4, the next step was to define an appropriate similarity threshold. In this context, the threshold refers to the minimum similarity score above which two product names are considered a valid match.

5.3.1 Jaro-Winkler and Similarity Threshold

The Jaro-Winkler similarity score ranges from 0 to 1, with higher scores indicating greater similarity. A threshold must be chosen to determine the minimum score required for a match to be accepted. A higher threshold reduces the number of false positives[1]. It increases the precision [2]. Still, it may miss some accurate matches, resulting in false negatives[3]. Conversely, a lower threshold increases recall [4] but may lead to inaccurate pairings.

5.3.2 Precision vs. Recall Trade-off

To find an optimal balance, we evaluated the matching results on a test set using different threshold values. We used precision and recall to quantify performance. To combine these metrics, we calculated the F1-score, which is the harmonic mean of precision and recall [72].

K-fold Cross-Validation

To ensure that the threshold selection was not biased by a particular subset of products, we applied k-fold cross-validation. In this approach, the dataset was divided into five equally sized folds, with a fixed random seed (random_state = 42) to ensure reproducibility. Each fold served once as a test set, while the remaining $k - 1$ folds were used to evaluate different similarity thresholds.

[1] False positives occur when two different products are incorrectly matched.

[2] The proportion of correct matches among all predicted matches

[3] False negatives occur when two similar products are not matched, even though they should be.

[4] The proportion of correct matches identified among all actual matches

Although the system does not involve model training, the similarity threshold acts as a decision parameter that influences the system's behaviour. Selecting this threshold based on a single evaluation set could lead to overfitting, particularly given the dataset's limited size. Cross-validation helps mitigate this risk by testing each threshold on multiple, distinct subsets of data. This provides a more reliable and generalizable estimate of performance, ensuring that the chosen threshold is not overly tailored to a single group of products but rather reflects realistic usage scenarios.

F1-score Optimisation

Our objective is to optimise the F1-score. Therefore, we chose to impose a minimum precision constraint, currently set at 90%. This threshold ensures that most accepted matches are reliable. However, this value remains arbitrary for now. To define it more rigorously, we would need a test set containing products with known, verified carbon footprints. This would allow us to compare our estimations with actual reference values and optimise the threshold not only based on classification metrics (such as F1-score) but directly on the estimation error of the carbon footprint.

Such an approach would enable us to identify the best compromise between two types of errors: those caused by using overly generic fallback values and those resulting from incorrect product matching. This would ultimately lead to more accurate and trustworthy carbon impact assessments.

In the following graph, the line represents the observed performance values, and the shaded area indicates the margin of error. We highlight the threshold that optimises the F1-score under the precision constraint with a vertical line.

As shown in Figure 5.4, the F1-score remains high and stable up to a threshold of 0.778. Beyond this point, it declines sharply and reaches 0% at a threshold of 1.0.

5.3.3 Analysis of Matching Limitations

Although we initially expected a match rate above 70%, the final result was lower than anticipated. In this section, we examine the primary factors that limit matching performance. These limitations are either structural (related to database content) or algorithmic (related to the matching strategy itself).

Limitations of High Similarity Scores

The sharp decline in precision and F1-score beyond the 0.78 threshold indicates that the algorithm struggles to differentiate between high and very high similarity scores. In other words, increasing the threshold does not improve the quality of

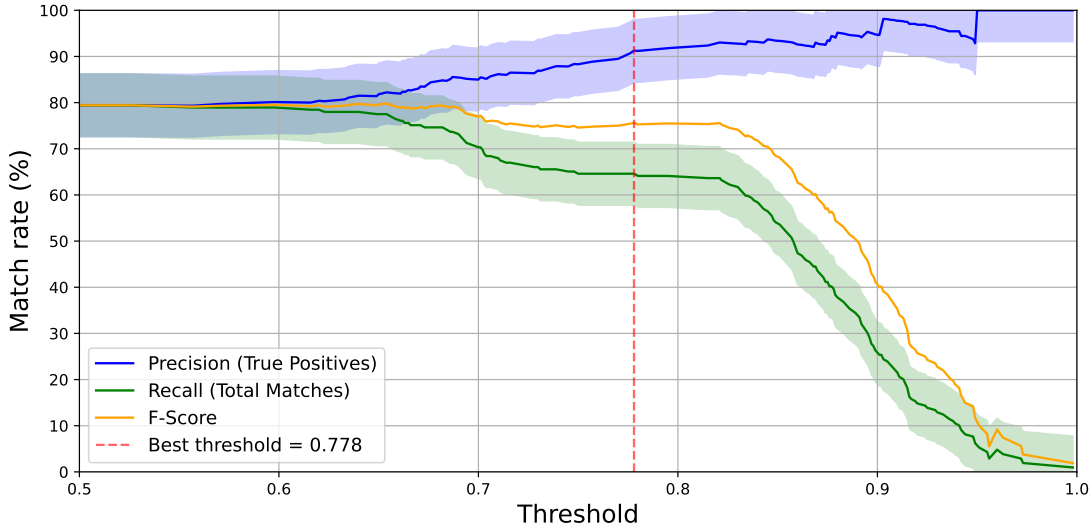

Figure 5.4: Evolution of precision, recall, and F1-score according to different threshold values.

the match; it only reduces the number of candidates, often without real semantic benefit.

One explanation is that some entries in the "Agribalyse" database are extremely similar. For instance, consider the Leclerc product *Chocolat Noir 74% De Cacao Bio – 100 g – BIO VILLAGE*. The algorithm must choose between two database items:

- *Chocolat noir à 70% cacao minimum, extra, dégustation, tablette*
- *Chocolat noir à moins de 70% de cacao, à croquer, tablette*

According to OpenFoodFacts [7], the first is correct. However, based solely on the name, the algorithm lacks sufficient information to distinguish between the two reliably. Even with a high similarity score, the match may be incorrect because the decision lacks contextual understanding.

In summary, the algorithm performs well at rejecting clearly unrelated candidates when the score is low. However, the model cannot always identify the best one when multiple close candidates have similar high scores, as it lacks semantic discrimination.

Impact of Filtering Errors

Another contributing factor is the candidate refinement process subsection 3.3.1, which reduces the number of candidates before applying the similarity score. If the correct match is accidentally removed during filtering, the algorithm cannot recover it. In that case, it may select the best match among the remaining items, even if none of them is correct. This situation can produce a false positive despite a seemingly good similarity score.

5.4 Algorithm Selection

This section explains the criteria and results that guided the choice of the final matching algorithm used in the system.

5.4.1 Best-Performing Approach

In this section, we explain why we chose this approach for matching and estimating carbon footprints. Our decision was based on both the performance and execution speed of the tested algorithms.

From the results of the previous tests, we found that combining data stemmatization with filtering provided the best results in terms of both matching accuracy and execution time. Stemmatization helps reduce words to their root form, while filtering quickly eliminates irrelevant comparisons, which is crucial for improving efficiency. This combination optimises both the quality and speed of the matching process, making it particularly well-suited to our use case, where processing time is a critical factor.

We also chose the Jaro-Winkler algorithm to match products. Among all the algorithms tested, it showed the highest overall matching rate. This algorithm excels at handling minor typographical errors and variations in string lengths, which are common in product names on receipts. It is especially effective when matching short strings, such as product names on receipts, where minor differences can significantly impact the quality of the match. Additionally, Jaro-Winkler adjusts the score based on the common prefix between two strings, which is helpful for product names that often start with brand names or categories, as is common in retail environments.

While other similarity algorithms, like Cosine, Dice, and Jaccard, were also tested, none consistently outperformed Jaro-Winkler in terms of accuracy, especially when filtering was applied. For instance, although Cosine performs well when the strings are noise-free, its performance drops when noise is present in the data, as is often

the case with receipts that are missing details or contain inconsistencies. In contrast, Jaro-Winkler maintains high accuracy due to its ability to handle frequent errors and inconsistencies in product names.

Therefore, combining Jaro-Winkler with data stemmatization and filtering turns out to be the most effective approach for this task, both in terms of precision and execution speed.

5.4.2 Conclusions from Previous Subsections

The choice of this algorithm is also supported by the results of the tests conducted in previous sections. For example, in Section subsection 2.2.3, we demonstrated that data preprocessing, including stemmatization and filtering, plays a crucial role in enhancing matching accuracy. By applying these techniques, we were able to reduce noise and improve the match rate by eliminating irrelevant items from the comparison set, thereby reducing the cost in terms of both time and computational resources. Filtering removed a significant number of irrelevant entries, which provided a substantial boost to the algorithm, particularly for token-based similarity measures such as Jaccard and Dice. Without filtering, these algorithms achieved match rates below 13%, highlighting the importance of having a solid preprocessing step.

Moreover, the choice of Jaro-Winkler was validated by the threshold calibration tests presented in Section 5.3. The analysis of precision versus recall showed that Jaro-Winkler, with an optimised threshold of 0.78, achieved a high accuracy of 91.78% while maintaining a reasonable recall of 64.11%. This balanced approach reduces false positives, which is particularly important for carbon footprint estimation, where incorrect product matches could lead to significant errors in environmental impact assessment. Additionally, the algorithm's ability to handle minor errors and variations in product names is key, as receipt data can vary significantly from one store to another.

In summary, the combination of preprocessing techniques and the Jaro-Winkler algorithm ensures optimal matching precision, providing a solid foundation for reliable carbon footprint estimates.

Chapter 6

Use Case Demonstration

In this section, we illustrate the program's operation using a typical purchase that demonstrates its main functionalities. We will execute the program multiple times on the same item, each time varying the available input information. This approach highlights how the precision and variation of the carbon footprint estimation depend on the initial data provided to the system.

6.1 Input Structure and Scenarios

The program takes as input a JSON file that contains a list of items grouped under a "ticket" object. Below is the example we will use as a base:

```
{
    "store": "Delhaize Auderghem",
    "date": "2023-03-20",
    "total": 4.29,
    "items": [
        {
        "barcode": null,
        "name":  "Fraise barquette (500 g)",
       ""pric"": null,
       ""quantit"": null,
       ""origin"": null,
       ""destinatio"": null,
       ""saiso"": null,
       ""Co"": null,
       ""packagin"": null
        }
    ]
}
```


Figure 6.1: example of input JSON structure containing only the barcode

This example includes a receipt containing a single item: a 400g package of strawberries. Each key in the item object represents a piece of information that may or may not be available to improve the accuracy of the carbon footprint estimation. The more information that is available, the more precise the estimate will be.

In our case, we only have the product name and its price. The goal is to deduce as much relevant information as possible from this input and produce the best possible carbon footprint estimate.

6.2 Case 1: Minimal Information

Here, we will take the input mentioned in section 6.1 and present the results our program provided, structured in a table. As we can see, even with only the product name and its weight, the program can produce a basic carbon footprint estimate. This is made possible by the matching process with the "Agribalyse" database, which identified the CIQUAL code 13014[1].

[1]The CIQUAL code is a unique identifier used in the French CIQUAL nutritional database, linked to the "Agribalyse" database, and designates a specific type of food product.

Input Information	Value
Barcode	None
Product Name	Fraise barquette (500 g)
Origin	None
Weight	0.5 kg
Packaging	None
CIQUAL Code	13014
Carbon Footprint	**Value**
CO_2 - Agriculture	0.120 kg
CO_2 - Transportation	0.135 kg
CO_2 - Distribution	0.0066 kg
CO_2 - Consumption	0.005 kg
CO_2 - Packaging	0.000 kg
Total CO_2	**0.2666 kg**

Table 6.1: Carbon footprint estimation with product name, weight, and CIQUAL code

In "Agribalyse", CIQUAL code 13014 refers to "Strawberry, raw", which accurately matches our item. Since we also know the product's weight, the program can compute a base estimate by multiplying this weight by the average carbon emission value, resulting in an initial footprint that does not yet reflect the product's specific characteristics.

However, because we lack information on the product's origin and packaging, it is impossible to refine the estimation by calculating transport emissions, seasonality effects, or packaging impact. Additionally, as no barcode is available, we could not use OpenFoodFacts to enrich the data, leaving us with a still fundamental carbon footprint estimate.

6.3 Case 2: Barcode Only

Here is the same example of input of the same strawberry item, but this time we only have the barcode of the product, not the name or anything else:

```
{
    "store": "Delhaize Auderghem",
    "date": "2023-03-20",
    "total": 4.29,
    "items": [
        {
        "barcode": 726061634,
        "name": null,
        "price": null,
        "quantity": null,
        "origine": null,
        "destination": null,
        "saison": null,
        "Co2": null,
        "packaging": null
        }
    ]
}
```

Figure 6.2: example of input JSON structure containing only the barcode

After passing this input into the program, we collect this information and represent it in a table

Table 6.2: Carbon footprint estimation with product name, weight, and CIQUAL code

Input Information	Value
Barcode	26061634
Product Name	Fraise barquette (500 g)
Origin	None
Weight	0.5 kg
Packaging	all: {}, en:plastic: {}, en:unknown: {}
CIQUAL Code	13014
Carbon Footprint Details	**Value**
CO_2 - Agriculture	0.120 kg
CO_2 - Transportation	0.135 kg
CO_2 - Distribution	0.0066 kg
CO_2 - Consumption	0.0050 kg
CO_2 - Packaging	0.0000 kg
Total CO_2	**0.2666 kg**

Table 6.3: Carbon footprint estimation with product name, weight, and CIQUAL code

Here, when examining the final result, we observe that it is identical to the case where only the product name was available. This is because the barcode was successfully matched with OpenFoodFacts, which allowed us to retrieve information about the product name and weight, but not about the packaging or the origin.

Although it is not explicitly shown in the table, OpenFoodFacts also lacked data regarding the product's carbon footprint. However, since we had the product name, we were able to use our matching algorithm to identify the corresponding CIQUAL code and retrieve the associated carbon footprint information.

As a result, the final estimate remains the same as in the case with only a name input, which is 0.2666 kg of CO_2.

Next, we will examine the changes that occur when all information is available to better understand how the final estimation improves with complete data.

6.4 Case 3: Complete Input

In this case, we have manually completed all the information related to the product to assess how the estimation changes when the program has access to all necessary data to compute the most accurate result possible. This situation could also occur when OpenFoodFacts contains all of these details in its database.

```
{
    "store": "Delhaize Auderghem",
    "date": "2023-03-20",
    "total": 4.29,
    "items": [
        {
        "barcode": 726061634,
        "name": "Fraise barquette (500 g)",
        "price": 4.29,
        "quantity": 0.5,
        "origine": "belgium",
        "destination": "France",
        "saison": null,
        "Co2": null,
        "packaging":
            {
            "all": {"weight": 6.89},
            "en:plastic": {"weight": 3.35},
            "en:unknown": {"weight": 3.54}
            }
        }
    ]
}
```


Figure 6.3: Example of a complete input JSON with full information available

Once the program executes, here is the result in a table form

As shown in Table Figure 6.4, the result has now changed. The CIQUAL code has been updated to 13014_3, instead of 13014, indicating that the data corresponds to off-season strawberries. This refinement was only possible because we now have access to the product's origin. Since the origin is France, the program inferred that the strawberries were likely grown in a cold-climate country outside the regular growing season. As the program was executed on May 23, 2025, this date falls outside the strawberry season according to the "Base Empreinte" database [8].

For transportation, knowing the country of origin now enables the program to calculate the transport route and the CO_2 emissions associated with each freight segment. Distribution and consumption values remain unchanged, as they are not

Input Information	Value
Barcode	26061634
Product Name	Fraise barquette (500 g)
Origin	Belgium
Weight	0.5 kg
Packaging	6.89 g (Plastic: 3.35 g, Unknown: 3.54 g)
CIQUAL Code	13014_3
Carbon Footprint Details	**Value**
CO_2 - Agriculture	0.1375 kg
CO_2 - Transportation	0.1350 kg
CO_2 - Distribution	0.0066 kg
CO_2 - Consumption	0.0050 kg
CO_2 - Packaging	0.0158 kg
Total CO_2	**0.2841 kg**

Table 6.4: Carbon footprint estimation with enriched metadata and updated agricultural value

currently impacted by input data in the present version of the model. Finally, the packaging emissions can now be calculated more precisely, replacing the default estimate.

All these refinements combined (see Section section 4.1) result in a total footprint of 0.2841 kg of CO_2 (Table Figure 6.4), which is slightly higher than the less detailed estimates presented earlier in Sections section 6.2 and section 6.3.

6.5 Case 4: Price Only

We will now examine the impact on the estimation when neither the product name nor the barcode is available, or when both fail to match any entries in the reference databases. In this case, only the product's price is known, which prevents the system from retrieving a baseline carbon footprint from environmental data sources.

```
{
    "store": "Delhaize Auderghem",
    "date": "2023-03-20",
    "total": 4.29,
    "items": [
        {
        "barcode": null,
        "name": null,
        "price": 4.29,
        "quantity": null,
        "origine": null,
        "destination": null,
        "saison": null,
        "Co2": null,
        "packaging": null
        }
    ]
}
```

Figure 6.4: example of an input JSON with only the price

Once the program executes, the table Figure 6.5 gives us the output of the program.

As shown in Table Figure 6.5, we have minimal information to analyse. Nevertheless,

Input Information	Value
Barcode	None
Product Name	None
Origin	None
Weight	None
Packaging	None, en:unknown: {}
CIQUAL Code	None
Carbon Footprint Details	**Value**
CO_2 - Agriculture	na
CO_2 - Transportation	na
CO_2 - Distribution	na
CO_2 - Consumption	na
CO_2 - Packaging	na
Total CO_2	**2.38524kg**

Table 6.5: Carbon footprint estimation with product name, weight, and CIQUAL code

the system still produces an estimated 2.3852 kg of CO_2, which is exceptionally high.

As explained in Section section 4.2, this is due to the fallback method where the program multiplies the product's price by an average emission factor, expressed in CO_2 per euro, representative of food items sold in supermarkets. Since strawberries generally have a lower carbon footprint compared to many processed products [22], this method tends to overestimate emissions when no specific product information is available.

If the price is also missing, the program is unable to perform any calculation and returns an error.

6.6 Results Comparison Summary

To better understand the influence of available input data on the accuracy of the carbon footprint estimation, the following table summarises all four test cases. It highlights the main fields provided, the matched CIQUAL code when applicable, and the final estimated CO_2 value.

Table 6.6: Comparison of carbon footprint estimations across test cases

Case	Available Data	CIQUAL Code	Total CO_2 (kg)
Case 1	Product name only	13014	0.2666
Case 2	Barcode only	13014	0.2666
Case 3	All fields (manual)	13014_3	0.2841
Case 4	Price only	None	2.3852

From this Comparison, we clearly observe that the completeness of input data significantly influences the accuracy of the carbon footprint estimation. In Cases section 6.2 and section 6.3, the estimates are identical due to successful enrichment via name or barcode, even in the absence of other data. In Case section 6.4, detailed information such as origin and packaging allows for refinement, slightly increasing the total. Conversely, Case Figure 6.5 exhibits a strong overestimation due to the fallback monetary method, which is used when no product-specific information is available.

Chapter 7

Limits and Future Work

This work faces key limitations, particularly concerning the accuracy of the matching algorithm. Despite significant efforts, the algorithm does not guarantee perfect precision, as evidenced by a match rate of approximately 64% at a threshold of 0.79 (Figure 5.4). This error rate may occasionally lead to incorrect matches, which in turn can distort carbon footprint estimations, especially when mismatched products are entirely unrelated.

A potential improvement would be to explore more sophisticated filtering algorithms. Currently, the filtering step relies solely on the number of common words between strings, which overlooks significant semantic similarities. Replacing this approach with advanced similarity metrics, such as Jaccard, Dice, or Cosine subsection 2.2.4, could improve the pre-filtering phase by prioritising candidates that are more contextually relevant. While these algorithms are less accurate in final matching due to limitations in precision, they can still be effective in reducing the number of candidates during early filtering.

Future work could involve designing a multi-stage matching pipeline. In this setup, advanced similarity metrics would be employed as an initial screening step, followed by a more precise algorithm, such as Jaro-Winkler subsection 2.2.4, for final validation. Such a layered approach could offer improvements in both accuracy and computational efficiency.

A second major limitation of the current work is the lack of complete product information. Although OpenFoodFacts section 2.2.1 is a rich database, it often lacks crucial data, such as weight, country of origin, and initial carbon footprint estimates. Furthermore, values from sources like Agribalyse section 2.2.1 are often national averages and do not reflect country-specific standards or practices. To address this limitation, future work should focus on incorporating more comprehensive

proprietary databases, such as EcoInvent section 2.2.1 or CarbonCloud section 2.2.1. Partnerships with these platforms could open access to updated and verified datasets, which are essential for ensuring the long-term relevance and credibility of the tool. This would enable the system to scale internationally, adapting estimations to reflect local agricultural practices, energy mixes, and regulatory standards.

Additionally, the current implementation cannot identify bulk products, which are typically absent from OpenFoodFacts [7]. This limitation restricts the extraction of accurate information. For some fresh items, packaging plays a significant role in the overall carbon footprint [73], and this gap may lead to inflated carbon estimates. Future work could involve analysing store barcode systems or utilising external studies to improve the identification of bulk products.

Lastly, future improvements should focus on the evaluation process by testing the system on real receipts. This would enable a more comprehensive analysis of the final results, the effectiveness of the matching algorithm, and the system's ability to retrieve missing information. These tests would provide valuable insights into the tool's behaviour under real-world conditions.

Chapter 8

Conclusion

This thesis presents the development of a method capable of automatically estimating the carbon footprint of food products based on grocery receipts. The goal is to offer consumers a better understanding of the environmental impact of their daily purchases.

The project is structured around several key components. First, a text normalisation and preprocessing pipeline has been implemented to extract relevant information from receipts, despite their often noisy and incomplete format. Then, a product matching system has been developed to link receipt items with environmental databases such as "Agribalyse", OpenFoodFacts, and "Base Empreinte". To enhance accuracy, natural language processing tools and similarity algorithms, including the Jaro-Winkler algorithm, are utilised in conjunction with filtering and linguistic standardisation techniques.

With the product match done, the system estimates the carbon footprint by considering the main life cycle stages: agriculture, transportation, and packaging. In the absence of specific information, fallback mechanisms using monetary emission factors ensure a plausible estimate. The system's performance was then evaluated through simulated use cases, showing how data completeness influences the accuracy of the results.

The findings demonstrate that a more tailored estimation of carbon footprint is feasible compared to existing tools, although certain limitations remain. Incomplete product data, particularly for bulk items, and the lack of testing on real receipts still limit the system's scope. Several areas for improvement have been identified, such as integrating proprietary databases, deploying more advanced matching models, and validating the system with real-world examples.

In conclusion, this work represents a first step toward an operational tool for carbon footprint estimation based on accessible and automated processes. It lays the groundwork for future research aimed at increasing the precision, robustness, and impact of consumer-oriented environmental decision-making.

Bibliography

[1] NASA, *The causes of climate change*, Consulté le 19 mai 2025, 2024. [Online]. Available: https://science.nasa.gov/climate-change/causes/.

[2] C. Bock and C. Böhler, "How to calculate the carbon footprint of your products," *Roland Berger*, 2023. [Online]. Available: https://www.rolandberger.com/en/Insights/Publications/How-to-calculate-the-carbon-footprint-of-your-products.html.

[3] "Eco-score – méthodologie de calcul." Accessed May 2025. (2021), [Online]. Available: https://docs.score-environnemental.com/.

[4] Greenly. "What is the greenhouse gas protocol?" Consulté le 19 mai 2025. (2023), [Online]. Available: https://greenly.earth/en-us/blog/company-guide/what-is-the-greenhouse-gas-protocol.

[5] "Open food facts — base de données collaborative sur les produits alimentaires." Consulté le 19 mai 2025, Open Food Facts. (2025), [Online]. Available: https://fr.openfoodfacts.org/.

[6] ADEME, *Accès aux données – AGRIBALYSE*, Consulté le 4 mai 2025, 2024. [Online]. Available: https://doc.agribalyse.fr/documentation/utiliser-agribalyse/acces-donnees.

[7] OpenFoodFacts, *OpenFoodFacts Data – Access to Product Data*, Consulté le 4 mai 2025, 2024. [Online]. Available: https://world.openfoodfacts.org/data.

[8] ADEME, *Base empreinte® : Base de données publique officielle de facteurs d'émission et de jeux de données d'inventaire*, Consulté le 11 mai 2025, 2023. [Online]. Available: https://base-empreinte.ademe.fr/.

[9] Explosion, *Spacy: Industrial-strength natural language processing in python*, https://spacy.io, Accessed April 2025, 2024.

[10] O. Rozinek and J. Mareš, "Fast and precise convolutional jaro and jaro-winkler similarity," *IEEE FRUCT*, 2024. [Online]. Available: https://www.researchgate.net/publication/380360789_Fast_and_Precise_Convolutional_Jaro_and_Jaro-Winkler_Similarity.

[11] S. Hariharan and R. Srinivasan, "A comparison of similarity measures for text documents," *Journal of Information & Knowledge Management*, vol. 7, no. 1, pp. 1–8, 2008. DOI: 10.1142/S0219649208001889.

[12] International Organization for miscization, *Environmental management – life cycle assessment – principles and framework*, ISO, 2006. [Online]. Available: https://www.iso.org/misc/37456.html.

[13] International Organization for miscization, *Environmental management – life cycle assessment – requirements and guidelines*, ISO, 2006. [Online]. Available: https://www.iso.org/misc/38498.html.

[14] P. Roy and T. Nei, "A review of life cycle assessment (lca) on some food products," *Journal of Food Engineering*, vol. 90, no. 1, pp. 1–10, 2009. DOI: 10.1016/j.jfoodeng.2008.06.016. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0260877408002793.

[15] A. N. Menegaki and M. A. Abdelsalam, "The quantification of carbon footprints in the agri-food sector and its role in sustainability: A review," *Sustainability*, vol. 15, no. 21, p. 15 611, 2023. DOI: 10.3390/su152115611. [Online]. Available: https://www.mdpi.com/2071-1050/15/21/15611.

[16] International Organization for miscization, *Greenhouse gases — carbon footprint of products — requirements and guidelines for quantification*, Consulté le 19 mai 2025, ISO, 2018. [Online]. Available: https://www.iso.org/misc/71206.html.

[17] J. Čapla, P. Zajác, and J. Čurlej, "The current state of carbon footprint quantification and tracking in the agri-food industry," *SciFood*, vol. 19, pp. 110–127, 2025. [Online]. Available: https://www.researchgate.net/publication/389096906_The_current_state_of_carbon_footprint_quantification_and_tracking_in_the_agri-food_industry.

[18] British miscs Institution, *Pas 2050:2011 – specification for the assessment of the life cycle greenhouse gas emissions of goods and services*, Consulté le 19 mai 2025, BSI miscs Limited, 2011. [Online]. Available: https://shop.bsigroup.com/products/pas-2050-2011.

[19] ecoinvent Association, *Ecoinvent database v3.10*, Consulté le 19 mai 2025, ecoinvent Association, 2024. [Online]. Available: https://ecoinvent.org/.

[20] World Resources Institute and World Business Council for Sustainable Development, *Product life cycle accounting and reporting misc*, Consulté le 19 mai 2025, GHG Protocol Initiative, 2011. [Online]. Available: https://ghgprotocol.org/product-misc.

[21] ecoinvent Association, *Ecoinvent licenses and pricing*, Accessed: 2025-05-16, 2024. [Online]. Available: https://ecoinvent.org/licenses/.

[22] ADEME, *Agribalyse – données environnementales des aliments*, Consulté le 19 mai 2025, 2025. [Online]. Available: https://agribalyse.ademe.fr/app/aliments.

[23] *Base empreinte ademe*, https://base-empreinte.ademe.fr/, Consulté en 2024.

[24] European Commission, *Ilcd handbook: General guide for life cycle assessment - detailed guidance*, Consulté le 19 mai 2025, Joint Research Centre, Institute for Environment and Sustainability, 2010. [Online]. Available: https://eplca.jrc.ec.europa.eu/uploads/ILCD-Handbook-General-guide-for-LCA-detailed-guidance-12March2010.pdf.

[25] CarbonCloud. "Climatehub." Accessed: 2025-05-16. (2025), [Online]. Available: https://apps.carboncloud.com/climatehub/.

[26] R. Pramana, Debora, J. J. Subroto, A. A. S. Gunawan, and Anderies, "Systematic literature review of stemming and lemmatization performance for sentence similarity," in *2022 IEEE 7th International Conference on Information Technology, Information Systems and Electrical Engineering (ICITISEE)*, 2022, pp. 315–320. DOI: 10.1109/ICITDA55840.2022.9971451. [Online]. Available: https://www.researchgate.net/publication/366174263_Systematic_Literature_Review_of_Stemming_and_Lemmatization_Performance_for_Sentence_Similarity.

[27] *7 Top NLP Libraries For NLP Development [Updated]*, en, 2024-10. [Online]. Available: https://www.labellerr.com/blog/top-7-nlp-libraries-for-nlp-development/ (visited on 2025-04-20).

[28] *Spacy/fr_core_news_sm · Hugging Face*. [Online]. Available: https://huggingface.co/spacy/fr_core_news_sm (visited on 2025-04-21).

[29] *Spacy/fr_core_news_md · Hugging Face*. [Online]. Available: https://huggingface.co/spacy/fr_core_news_md (visited on 2025-04-21).

[30] *Spacy/fr_core_news_lg · Hugging Face*. [Online]. Available: https://huggingface.co/spacy/fr_core_news_lg (visited on 2025-04-21).

[31] F. N. Flores and V. P. Moreira, "Assessing the impact of Stemming Accuracy on Information Retrieval – A multilingual perspective," *Information Processing & Management*, vol. 52, no. 5, pp. 840–854, 2016-09, ISSN: 0306-4573. DOI: 10.1016/j.ipm.2016.03.004. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0306457316300358 (visited on 2025-04-21).

[32] S. Hariharan and R. Srinivasan, "A Comparison of Similarity Measures for Text Documents," en, *Journal of Information & Knowledge Management*, vol. 07, no. 01, pp. 1–8, 2008-03, ISSN: 0219-6492, 1793-6926. DOI: 10.1142/S0219649208001889. [Online]. Available: https://www.worldscientific.com/doi/abs/10.1142/S0219649208001889 (visited on 2025-04-21).

[33] J. Andersson, "A comparison of different methods in their ability to compare semantic similarity between articles and press releases," en,

[34] A. Zeakis, D. Skoutas, D. Sacharidis, O. Papapetrou, and M. Koubarakis, "Tokenjoin: Efficient filtering for set similarity join with maximum weighted bipartite matching," *Proceedings of the VLDB Endowment*, vol. 16, no. 4, pp. 790–802, 2022. DOI: 10.14778/3574245.3574263. [Online]. Available: https://www.vldb.org/pvldb/vol16/p790-skoutas.pdf.

[35] HeiGIT gGmbH, *Openrouteservice api*, Accessed: 2025-05-19, 2025. [Online]. Available: https://openrouteservice.org.

[36] G. Halili, *Searoute: A python package to calculate the shortest sea route between two coordinates*, https://pypi.org/project/searoute/, Accessed on 2025-05-12, 2023.

[37] BNP Paribas, *Empreinte carbone — transition énergétique et écologique*, https://mabanque.bnpparibas/content/dam/mabanque/rsc/contrib/pdf/Notice-methodologique-Greenly-BNPP-BDDF.pdf, Consulté le 12 mai 2025, 2024.

[38] Lucas. "L'empreinte carbone des aliments." Consulté le 19 mai 2025, La Fourche. (2024-06), [Online]. Available: https://blog.lafourche.fr/empreinte-carbone-des-aliments.

[39] "La fourche – empreinte carbone des produits alimentaires." Accessed May 2025. (2023), [Online]. Available: https://lafourche.fr.

[40] Yuka, *Yuka – mobile app for food and cosmetic product analysis*, https://yuka.io, Accessed: 2025-05-24, 2025.

[41] M. Devillet, *Carbon calculator - github repository*, Accessed: 2025-05-04, 2025. [Online]. Available: https://github.com/menace31/Carbon-calculator.

[42] Impact CO2, *Fruits et légumes de saison*, https://impactco2.fr/outils/fruitsetlegumes, Accessed: 2025-05-04, 2025.

[43] Q. Zhu, B. Wang, Z. Lin, Y. Wu, C. Zhang, and X. Liu, "Is text preprocessing still worth the time? a comparative survey on modern text classification," *Information Processing & Management*, vol. 60, no. 6, p. 103 414, 2023. DOI: 10.1016/j.ipm.2023.103414. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0306437923001783.

[44] S. Bird, E. Loper, and E. Klein, *Natural language toolkit (nltk)*, https://www.nltk.org, Accessed April 2025, 2024.

[45] Delhaize, *Pommes Royal Gala Emballé 6pc*, https://www.delhaize.be/fr/shop/Fruits-et-legumes-frais/Fruits-frais/Pommes-et-poires/Pommes/Pommes-Royal-gala-Emballe-6pc/p/F2021060800081990000, Consulté en mai 2025, 2025.

[46] J. Wang, G. Li, and J. Feng, "Fast-join: An efficient method for fuzzy token matching based string similarity join," in *IEEE International Conference on Data Engineering (ICDE)*, 2011. DOI: 10.1109/ICDE.2011.5767865. [Online]. Available: https://sci-hub.st/10.1109/ICDE.2011.5767865.

[47] *Les emballages alimentaires - conseil national de l'alimentation*, https://cna-alimentation.fr/wp-content/uploads/2021/07/CNA-DP-emballages-alimentaires_juillet-2021.pdf, Juillet 2021.

[48] BNP Paribas, *Empreinte carbone — transition énergétique et écologique*, https://mabanque.bnpparibas/fr/vos-besoins/vos-projets/transition-energetique-et-ecologique/empreinte-carbone, Accessed on 2025-05-12, 2025.

[49] Green-Got, *Comment est calculée l'empreinte carbone des achats ?* Consulté le 12 mai 2025, 2024. [Online]. Available: %5Curl%7Bhttps://faq.green-got.com/impact/aJ2GPASSFuqSHiu5tVHBie/comment-est-calcul%C3%A9e-l%E2%80%99empreinte-carbone-des-achats--/7N41aukkzKqYFYkYiircMJ%7D.

[50] Collect & Go, *Nakd Blueberry Muffin 140g*, https://www.collectandgo.be/colruyt/nl/assortiment/nakd-blueberry-muffin-140g, Accessed: 2025-05-19, 2025.

[51] Agrobio Bretagne, *Tomates : Privilégier l'origine locale et la saison !* https://www.agrobio-bretagne.org/wp-content/uploads/2022/01/210429-CP-tomates.pdf, Communiqué de presse, 2021. [Online]. Available: https://www.agrobio-bretagne.org/wp-content/uploads/2022/01/210429-CP-tomates.pdf.

[52] B. Paris, F. Vandorou, A. T. Balafoutis, *et al.*, "Energy use in greenhouses in the eu: A review recommending energy efficiency measures and renewable energy sources adoption," *Applied Sciences*, vol. 12, no. 10, p. 5150, 2022. DOI: 10.3390/app12105150. [Online]. Available: https://www.mdpi.com/2076-3417/12/10/5150.

[53] C. Perpiña Castillo, B. Kavalov, I. Vandecasteele, and F. Batista e Silva, "Energy use in heated greenhouses across europe," *Energies*, vol. 16, no. 19, p. 6788, 2023. DOI: 10.3390/en16196788. [Online]. Available: https://www.mdpi.com/1996-1073/16/19/6788.

[54] M. de l'Agriculture et de la Souveraineté Alimentaire, *Prospective agriculture-énergie 2030: L'agriculture face aux défis énergétiques*, Consulté en mai 2025, 2023. [Online]. Available: https://agriculture.gouv.fr/prospective-agriculture-energie-2030-lagriculture-face-aux-defis-energetiques.

[55] *Impact carbone d'un bien de consommation*, https://www.carbone4.com/files/wp-content/uploads/2017/10/impact_carbone_GB_page_a_page.pdf, Carbone 4, consulté en 2024.

[56] La Bella Frutta, *Tableau de conservation des fruits et légumes*, Accessed May 17, 2025, 2025. [Online]. Available: https://www.labellafrutta.be/Fruits-Bio-Primeurs/conseils/tableau-de-conservation-des-fruits-et-legumes/.

[57] J. Poore and T. Nemecek, *Global food miles by transport method*, https://ourworldindata.org/grapher/food-miles-by-transport, Our World in Data, Global Change Data Lab, 2018. (visited on 2025-05-10).

[58] Eurostat, *Road freight transport by journey characteristics*, https://ec.europa.eu/eurostat/statistics-explained/index.php?title=Road_freight_transport_by_journey_characteristics, Consulté le 10 mai 2025, 2024.

[59] T. Mayer and S. Zignago, "Notes on cepii's distances measures: The geodist database," CEPII, Tech. Rep. 2011-25, 2011. [Online]. Available: https://www.cepii.fr/PDF_PUB/wp/2011/wp2011-25.pdf.

[60] La Bella Frutta, *Tableau de conservation des fruits et légumes*, Accessed: 2025-05-11, 2025. [Online]. Available: https://www.labellafrutta.be/Fruits-Bio-Primeurs/conseils/tableau-de-conservation-des-fruits-et-legumes/.

[61] Mapado, *Haversine - calculate the distance between two points on earth*, https://pypi.org/project/haversine/, Python package version 2.8.0, 2025.

[62] myclimate, *Bases de calcul - calculateur d'émissions de vol*, Consulté le 11 mai 2025, 2025. [Online]. Available: https://www.myclimate.org/fr-ch/sinformer/notre-profil/telechargements/calculateur-de-vol/.

[63] S. Incecik, *Short-haul flights emit more carbon than long-haul flights*, https://www.aa.com.tr/en/environment/short-haul-flights-emit-more-carbon-than-long-haul-flights/2920623, Consulté le 11 mai 2025, 2023.

[64] *Ctac - emballage alimentaire*, https://www.agrireseau.net/transformation-alimentaire/documents/ctac_emballage_alimentaire.pdf, AgriRéseau, consulté en 2024.

[65] R. Fogliato, D. Borkan, U. Bhatt, S. Garfinkel, S. Friedler, and W. J. Scheirer, "Confidence intervals for error rates in 1:1 matching tasks," *arXiv preprint arXiv:2306.01198*, 2024. [Online]. Available: https://arxiv.org/abs/2306.01198.

[66] James E. Bartlett, II Joe W. Kotrlik Chadwick C. Higgins, *Organizational research: Determining appropriate sample size in survey research*, https://www.opalco.com/wp-content/uploads/2014/10/Reading-Sample-Size1.pdf, Accessed: 2025-05-24, 2025.

[67] L. Naing, T. Winn, and B. N. Rusli, "Practical issues in calculating the sample size for prevalence studies," *Archives of Orofacial Sciences*, vol. 1, pp. 9–14, 2006. [Online]. Available: https://www.researchgate.net/publication/45459002_Practical_Issues_in_Calculating_the_Sample_Size_for_Prevalence_Studies.

[68] LimeSurvey, *A comprehensive guide to understanding margin of error*, Accessed: 2025-05-27, 2024. [Online]. Available: https://www.limesurvey.org/blog/tutorials/a-comprehensive-guide-to-understanding-margin-of-error.

[69] Z. Survey, *Margin of error – definition, usage, and calculator*, Accessed: 2025-05-27, 2024. [Online]. Available: https://www.zoho.com/survey/margin-of-error.html.

[70] SurveyLab, *Margin of error in surveys: What is it and how to tackle it?* Accessed: 2025-05-27, 2024. [Online]. Available: https://www.surveylab.com/blog/margin-of-error/.

[71] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," in *Journal of Machine Learning Research*, vol. 13, JMLR, 2012, pp. 281–305. [Online]. Available: https://www.jmlr.org/papers/volume13/bergstra12a/bergstra12a.pdf.

[72] Z. C. Lipton, C. Elkan, and B. Narayanaswamy, "Thresholding classifiers to maximize f1 score," *arXiv preprint arXiv:1402.1892*, 2014. [Online]. Available: https://arxiv.org/abs/1402.1892.

[73] National Zero Waste Council, "Food loss & waste: Answering the packaging question," 2020, Consulté le 19 mai 2025. [Online]. Available: https://foodpackagingforum.org/news/report-investigates-greenhouse-gas-emissions-packaging-and-food-waste.

Appendix

repositories

The code used in this thesis is available at https://github.com/menace31/Carbon-calculator.

UNIVERSITÉ CATHOLIQUE DE LOUVAIN
École polytechnique de Louvain
Rue Archimède, 1 bte L6.11.01, 1348 Louvain-la-Neuve, Belgique | **www.uclouvain.be/epl**